

ABSTRACT

Indoor navigation remains challenging for visually impaired (VI) individuals due to the complexity of indoor layouts. In this project, a QR code indoor navigation system is proposed to assist VI individuals navigate indoor environments independently, efficiently, and safely. OpenCV is used for colour processing to extract coloured QR codes in red, blue, or green, making them easier for VI users to identify. The ZBar QR code reader is then applied to accurately detect and scan coloured QR codes that contain user location data. After a scan, the system determined the user's position, computed the shortest route with the A* algorithm, and guided the user toward the next coloured QR code for localisation. The system features a simple interface that delivers real-time, step-by-step voice instructions. Each module was evaluated separately through functional and non-functional tests. Results showed a QR code detection success rate of **66.7%** at a range of **30 cm** with larger markers (**256 cm²**), a QR code decoding rate of **83.78%** aided by grayscale and binarization, and path planning with an optimality ratio of **1.000** and a median runtime of **1.505 ms**. The route guidance tests passed all left and right turning test cases, and input validation successfully handled the numeric, text, and invalid entries. The system was then tested in the University Tunku Abdul Rahman (UTAR), Faculty of Information and Communication Technology (FICT) Block N, which confirmed that the end-to-end integration operated smoothly with no notable latency across modules. These outcomes demonstrated a practical, low-cost navigation aid that supports safer, more independent mobility for VI users.

Area of Study: Indoor Navigation, Image Processing

Keywords: Visually Impaired, Colour Segmentation, QR Code, A* Path Planning, Route Guidance, Text to Speech

TABLE OF CONTENTS

TITLE PAGE	i
ABSTRACT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	vi
LIST OF TABLES	ix
LIST OF SYMBOLS	x
LIST OF ABBREVIATIONS	x
CHAPTER 1 INTRODUCTION	1
1.1 Problem Statement and Motivation	1
1.2 Project Scope and Objectives	1
1.3 Impact, Significance and Contribution	2
1.4 Background Information	3
1.5 Report Organization	4
CHAPTER 2 LITERATURE REVIEW	5
2.1 Review of Survey Literature on Vision-based Indoor Navigation	5
2.2 Colour Processing for QR Code Isolation in Indoor Environments	7
2.3 Review of QR Code Detection Algorithms	8
2.4 Review of Technologies and Approaches in Indoor Navigation	10
2.5 Comparison of Methods	13
2.5.1 Marker vs Marker-less Approach	13
2.5.2 Comparison of Colour Processing for QR Code Techniques	13
2.5.3 Comparison of QR Code Detection Techniques	14
2.5.4 Comparison of Navigation Techniques	15

CHAPTER 3 SYSTEM OVERVIEW AND DESIGN	17
3.1 Design Overview	17
3.2 System Design	18
3.2.1 QR Code Detection	18
3.2.2 QR Code Reader	20
3.2.3 Path Planning	21
3.2.4 Route Guidance	23
3.3 Implementation Issues and Challenges	24
 CHAPTER 4 SYSTEM IMPLEMENTATION	 26
4.1 Hardware Setup	26
4.2 Software Setup	26
4.2.1 Development Environment and Programming Language	26
4.2.2 Software Specification	27
4.2.3 Input Source Setup	28
4.2.4 File Operation and Data Storage Setup	28
4.3 System Operation	29
4.3.1 Input Section	29
4.3.2 System Display	29
4.4 Concluding Remark	31
 CHAPTER 5 SYSTEM EVALUATION AND DISCUSSION	 32
5.1 System Testing and Performance Metrics	32
5.1.1 QR Code Detection	32
5.1.2 QR Code Reader	34
5.1.3 Path Planning	36
5.1.4 Route Guidance Testing	37
5.1.5 Input Validation Testing	38
5.2 Testing Setup and Result	39
5.2.1 QR Code Detection	39
5.2.2 QR Code Reader	44
5.2.3 Path Planning	46

5.2.4	Route Guidance Testing	49
5.2.5	Input Validation Testing	52
5.3	Project Challenges	54
5.4	Objectives Evaluation	55
5.5	Concluding Remark	56
CHAPTER 6 CONCLUSION AND RECOMMENDATIONS		57
6.1	Conclusion	57
6.2	Future Considerations and Recommendations	58
BIBLIOGRAPHY/REFERENCES		60
APPENDIX A		
A.1	App Controller Source Codes	A-1
A.2	QR Code Detection Source Codes	A-10
A.3	QR Code Reader Source Codes	A-22
A.4	Map Creation and Path Planning Source Codes	A-25
APPENDIX B		
	Plagiarism Check Result	B-1

LIST OF FIGURES

Figure Number	Title	Page
Figure 2.1.1	Breakdown of Indoor Navigation Systems [3]	5
Figure 2.1.2	Classification of Methods and Systems in Vision-Based Approaches [4]	6
Figure 2.2.1	Process of K-means Clustering in [7]	7
Figure 2.2.2	Process of Region Growing in [6]	8
Figure 2.3.1	QR Code Detection Process of [8]	9
Figure 2.3.2	QR Code Recognition Process in [9]	10
Figure 2.3.3	Image Preprocessing Process in [9]	10
Figure 2.4.1	Navigation with Real-time Obstacle Avoidance [10]	11
Figure 2.4.2	Navigation System Functional Block Diagram [10]	11
Figure 2.4.3	Architecture of The Navigation System [11]	12
Figure 3.1.1.1	System Overview	17
Figure 3.2.1.1	QR Code Detection Module	18
Figure 3.2.2.1	QR Code Reader Module	20
Figure 3.2.3.1	Path Planning Module	21
Figure 3.2.4.1	Route Guidance Module	23
Figure 4.3.1.1	Inserting Target Colour	29
Figure 4.3.1.2	Inserting Destination	29
Figure 4.3.2.1	System Partitions	30
Figure 4.3.2.2	Cropped QR Code Partition	30
Figure 4.3.2.3	Text Information Partition	31
Figure 5.1.2.1	QR Code Detected Images	34
Figure 5.1.2.2	QR Code Images Under Bright Environment	34
Figure 5.1.2.3	QR Code Images Under Low Environment	35
Figure 5.1.2.4	QR Code Images with Uneven Brightness	35
Figure 5.2.1.1	QR Code Attached on Door Glass	40
Figure 5.2.1.2	QR Code with Triangle Prism Structure	40
Figure 5.2.1.3	QR Code Attached on Door Panel	41
Figure 5.2.1.4	Example of Real-world Testing and Result	42

Figure 5.2.1.5	Comparison of QR Codes Between High-quality and Low-quality Print	43
Figure 5.2.2.1	Total QRs Failed and Successfully Decoded from Original, Gray, and Binarized Images	44
Figure 5.2.2.2	Confusion Matrix of QR Code Decoding Results	44
Figure 5.2.2.3	Example of QR Code Decoded on Original Image	45
Figure 5.2.2.4	Example of QR Code Decoded on Gray Image	45
Figure 5.2.2.5	Example of QR Code Decoded on Binarized Image	46
Figure 5.2.2.6	Example of QR Code Failed to Decode	46
Figure 5.2.3.1	Overview of The Route Sets	47
Figure 5.2.3.2	Optimality Statistics on Route Sets	47
Figure 5.2.3.3	Efficiency Results on Route Sets	48
Figure 5.2.3.4	Visualization of Evaluation on Route Sets	48
Figure 5.2.4.1	Output for ITC-001	50
Figure 5.2.4.2	Output for ITC-002	50
Figure 5.2.4.3	Output for ITC-003	50
Figure 5.2.4.4	Output for ITC-004	50
Figure 5.2.4.5	Output for ITC-005	50
Figure 5.2.4.6	Output for ITC-006	50
Figure 5.2.4.7	Output for ITC-007	50
Figure 5.2.4.8	Output for ITJ-001	51
Figure 5.2.4.9	Output for ITJ-002	51
Figure 5.2.4.10	Output for ITJ-003	51
Figure 5.2.4.11	Output for ITJ-004	51
Figure 5.2.4.12	Output for ITJ-005	51
Figure 5.2.4.13	Output for ITJ-006	51
Figure 5.2.4.14	Output for ITJ-007	51
Figure 5.2.5.1	Output for IT-012	53
Figure 5.2.5.2	Output for IT-013	53
Figure 5.2.5.3	Output for IT-014	53
Figure 5.2.5.4	Output for IT-015	54
Figure 5.2.5.5	Output for IT-016 (Invalid Room ID)	54

Figure 5.2.5.6	Output for IT-016 (Numeric Value)	54
Figure 5.2.5.7	Output for IT-016 (Symbols)	54

LIST OF TABLES

Table Number	Title	Page
Table 2.5.1	Comparison between Marker-Based and Markerless Approach	13
Table 2.5.2	Comparison of Colour Processing Techniques	13
Table 2.5.3	Comparison of QR Code Detection Techniques	14
Table 2.5.4	Comparison of Navigation Techniques	15
Table 4.1.1	Specification of Processing Unit	26
Table 4.1.2.1	Description of Visual Studio Code	26
Table 4.2.2.1	Software Specification	27
Table 4.2.3.1	Input Sources Description	28
Table 5.1.1.1	Robustness Test Variables for the QR Code Detection Module	33
Table 5.1.1.2	Performance Metrics Summary for the QR Code Detection Module	33
Table 5.1.2.1	QR Code Reader Module Evaluation Details	35
Table 5.1.4.1	Turn Right Navigation Instruction Test	37
Table 5.1.4.2	Turn Left Navigation Instruction Test	38
Table 5.1.5.1	QR Code Input Test	38
Table 5.1.5.2	Destination Room ID Input Test	39
Table 5.1.5.3	Scanned QR Code Input Test	39
Table 5.2.1.1	Detection Performance Under Different Conditions	41
Table 5.2.4.1	Turn Right Navigation Instruction Results	49
Table 5.2.4.2	Screenshot on Turn Right Navigation Instructions	50
Table 5.2.4.3	Turn Left Navigation Instruction Results	50
Table 5.2.4.4	Screenshot on Turn Left Navigation Instructions	51
Table 5.2.5.1	QR Colour Input Test Result	52
Table 5.2.5.2	Destination Room ID Input Test	52
Table 5.2.5.3	Scanned QR Code Input Test	53
Table 5.2.5.4	Scanned QR Code Actual Output	53

LIST OF SYMBOLS

A algorithm* A star algorithm

LIST OF ABBREVIATIONS

<i>VI</i>	Visually Impaired
<i>QR</i>	Quick Response
<i>YOLO</i>	You Only Look Once
<i>A* algorithm</i>	A star algorithm
<i>GPS</i>	Global Positioning System
<i>UI</i>	User Interface
<i>AR</i>	Augmented Reality
<i>WiFi</i>	Wireless Fidelity
<i>IMU</i>	Inertial Measurement Unit
<i>2D</i>	Two Dimensions
<i>3D</i>	Three Dimensions
<i>SLAM</i>	Simultaneous Localization and Mapping
<i>SfM</i>	Structure from Motion
<i>CNN</i>	Convolution Neural Networks
<i>HSV</i>	Hue, Saturation, and Value
<i>RGB</i>	Red, Green, Blue
<i>HS</i>	Hue-Saturation
<i>HV</i>	Hue-Value
<i>SV</i>	Saturation-Value
<i>BP</i>	Backpropagation neural network
<i>CAD</i>	Computer-Aided Design
<i>RGB-D</i>	Red, Green, Blue Depth
<i>TSM</i>	Time-Stamped Map
<i>SVD</i>	Singular Value Decomposition
<i>ADF</i>	Area Description File
<i>KF</i>	Kalman Filter
<i>HMI</i>	Human-Machine Interface
<i>SURF</i>	Speeded-Up Robust Features
<i>AI</i>	Artificial Intelligence
<i>OSTU algorithm</i>	Ōtsu Nobuyuki
<i>OS</i>	Operating System

<i>CPU</i>	Central Processing Unit
<i>GPU</i>	Graphics Processing Unit
<i>RAM</i>	Random-Access Memory
<i>RTX</i>	Ray Tracing Texel
<i>GDDR6</i>	Graphics Double Data Rate 6
<i>DDR5</i>	Double Data Rate 5
<i>SSD</i>	Semiconductor Storage Device
<i>RSSI</i>	Received Signal Strength Indicator
<i>UTAR</i>	University Tunku Abdul Rahman
<i>FICT</i>	Faculty of Information and Communication Technology
<i>P</i>	Progressive Scan

CHAPTER 1

Introduction

1.1 Problem Statement and Motivation

Individuals with visual impairments often struggle with indoor navigation. Although outdoor navigation technologies such as the Global Positioning System (GPS) are well-established, they are not suitable for indoor environments due to weak signal reception. According to [1], indoor navigation remains a significant challenge for visually impaired (VI) individuals because of the lack of a reliable and accessible wayfinding solution in a clustered indoor environment. Existing indoor navigation systems commonly have drawbacks, such as being too costly, lacking real-time updates and dynamic obstacle detection, or not providing intuitive guidance tailored to their needs [2].

Therefore, this project aims to develop an advanced and affordable vision-based system to assist VI individuals with indoor navigation. By leveraging computer vision and the smartphone's built-in camera, the system enables accurate positioning and guidance. The successful implementation of this solution would enhance safety, improve accessibility, and promote greater independence for guiding VI individuals in indoor environments.

1.2 Project Scope and Objectives

Project Scope

This project focuses on developing an indoor navigation system for the visually impaired (VI) within the Faculty of Information and Communication Technology (FICT) building at Universiti Tunku Abdul Rahman (UTAR). This project uses coloured QR codes as location markers, which are placed on door glass panels throughout the building. VI will be directed to the nearest QR code location to scan the QR code with a camera-enabled device. The current location of VI will be determined, and navigation guidance will be provided based on the shortest path calculated for VI to reach the destination.

CHAPTER 1

The scope of this project is limited to the ground floor of the FICT building, assuming that pathways are free of obstacles. The project will not cover outdoor navigation, GPS integration, real-time obstacle detection, or deployment in other buildings.

Project Objectives

- i. To develop an indoor navigation system for the visually impaired (VI) using coloured QR codes to determine location and guide them to their destination
- ii. To integrate modules, algorithms, and APIs, including QR code detection, A* pathfinding, and text-to-speech for accurate, accessible, and seamless navigation.

1.3 Impact, Significance and Contribution

This study addresses navigation challenges encountered by the visually impaired (VI), for whom moving around indoors in an unfamiliar environment without visual cues can be challenging. This often results in heavy reliance on others and potential safety hazards. By implementing a robust and inexpensive indoor navigation system using QR codes, VI users can receive real-time, step-by-step audio and haptic instructions, allowing them to reach their destinations independently and safely. This approach enhances personal safety, mobility and promotes equal access to public spaces for all members of society.

This approach is important because it provides an affordable indoor navigation solution. Traditional solutions often require expensive and sophisticated methodologies which are not easily adoptable. Our QR code-based solution addresses this shortcoming by leveraging the ubiquity of smartphone technology and a simple, cost-effective marker system.

Our key contribution lies in the development of a fully integrated, user-centric system . This includes a simple user interface, text-to-speech feature, and an accessible route planning to transform indoor environments into spaces where VI individuals can navigate freely and confidently.

1.4 Background Information

Indoor navigation has garnered attention in recent years due to its broad applications in surveillance, robotics, Augmented Reality (AR), and assistive navigation systems [3, 4]. This surge in interest is driven by the increasing ubiquity of smartphones and other wireless devices [5]. According to [5], indoor localisation refers to determining the position of a device or individual within an enclosed environment. Although the GPS excels in outdoor navigation, it often fails in indoor navigation [4]. This limitation poses significant challenges for visually impaired (VI) individuals, who rely on accurate and accessible guidance to navigate unfamiliar spaces safely.

To overcome the limitations of GPS indoors, solutions such as wireless technologies – WiFi, Bluetooth beacons, inertial sensors, and computer vision – have been employed [3, 4]. Indoor localisation systems can be generally classified as range-based and range-free approaches, each with its own balance of accuracy, cost, and scalability [5]. Hybrid systems that combine multiple technologies often achieve better performance, particularly in complex environments [3, 5]. Vision-based methods, especially marker-based approaches, are valued for their low cost and ease of deployment, though they have their own set of challenges – lighting variability and occlusion [4]. For VI users, system design must prioritise safety, reliability, and ease of use, ensuring consistent performance across diverse indoor settings [3].

1.5 Report Organization

Done by: Chong Jia Hui

This report is structured into six chapters covering the development, implementation, and evaluation of an indoor navigation system for the visually impaired (VI) using QR codes.

Chapter 1 introduces the project by outlining the problem statement, motivation, project scope, specific objectives, and background.

Chapter 2 presents a comprehensive literature review, including the survey literature on vision-based indoor navigation, colour processing for QR code isolation, QR code detection algorithms, and indoor navigation technologies. It highlights the strengths and limitations of existing solutions and establishes the foundation for the framework for this project.

Chapter 3 details the system methodology, providing a system overview and design. It describes the overall system architecture using a flowchart and explains each component with its functionalities.

Chapter 4 discusses the implementation processing, including the hardware and software setup. It also describes the system operation from a user's perspective.

Chapter 5 focuses on system evaluation, where the performance, accuracy, and efficiency of the implemented framework are assessed through various testing, such as functional and non-functional testing. The results are analysed to validate the system's effectiveness in meeting the predefined objectives. The challenges encountered during the analysis and testing are also discussed.

Lastly, Chapter 6 concludes the report with a summary of key achievements, reflections on project limitations, and proposals for future enhancements. It emphasises the project's contributions and its applicability to real-world indoor navigation.

CHAPTER 2: Literature Reviews

2.1 Review of Survey Literature on Vision-based Indoor Navigation

Vision-based solutions stand out for their precision, affordability, and compatibility with modern devices that already feature advanced cameras, further decreasing the costs and power consumption [3, 4]. Additionally, the chances of information delay are minimised [4]. These systems also provide the flexibility to integrate other technologies like Inertia Measurement Unit (IMU) and wireless signals, increasing accuracy [3, 4]. However, vision-based indoor navigation systems commonly face challenges: maintaining accuracy in varying lighting and managing object occlusions [3, 4]. Cameras, regardless of whether they are 2D or 3D, can struggle with untextured or reflective surfaces, affecting detection and depth estimation. Furthermore, balancing high computational costs with feature complexity and ensuring user comfort remains a persistent hurdle [3].

According to Figure 2.1.1, vision-based approaches are broadly categorised into marker-based and markerless methods. These systems are further classified by platform or application, such as robot navigation systems, wearable technology, AR visualization systems, and mobile device-based systems [4].

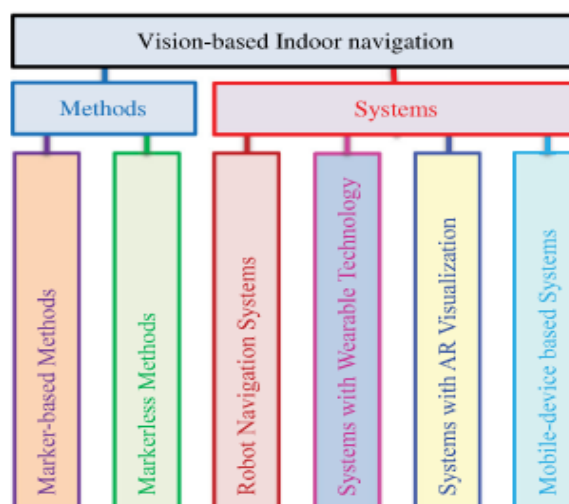


Figure 2.1.1: Breakdown of Indoor Navigation Systems [4]

CHAPTER 2

Based on Figure 2.1.2, all indoor positioning methods begin with an initial configuration stage to collect environmental data, such as physical marker positions, static camera locations, annotated image databases, or 3D models (via manual modeling, SLAM, or SfM). These methods are further distinguished by sensing devices —typically static or mobile 2D/3D cameras, potentially supplemented by WiFi/IMU data — and by their reliance on artificial markers or natural environmental features. Localisation techniques range from traditional image analysis to AI-based approaches [3]. Traditional methods detect features by comparison, while AI techniques, such as convolutional neural networks (CNNs), learn features directly from images, reducing the need for manual selection. However, AI approaches often require large training datasets, which can be a limitation due to the scarcity of publicly available 3D datasets [3]. These classifications support systematic analysis of diverse implementations and help identify common challenges and opportunities across application contexts [3, 4].

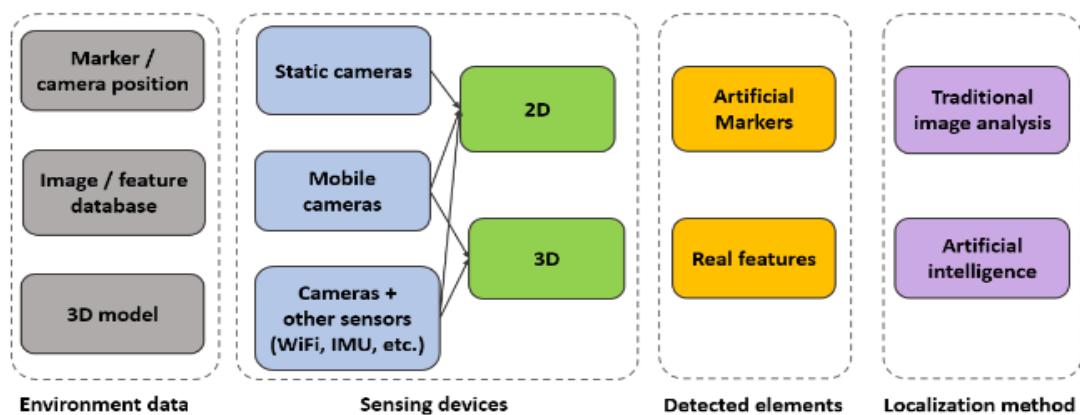


Figure 2.1.2: Classification of Methods and Systems in Vision-Based Approaches [3]

Vision-based systems often rely on handheld or wearable devices. These devices aid by providing audio and haptic feedback to the user. However, such implementations may impact user comfort and increase costs [4]. Marker-based and markerless methods vary in their advantages and disadvantages, as discussed in Section 2.5.

2.2 Colour Processing for QR Code Isolation in Indoor Environments

A key consideration for colour-based detection is the choice of colour space. In [6], the HSV colour space was utilised for its ability to separate brightness intensity from hue and saturation, making it more robust for colour-based segmentation compared to RGB. The target colour distribution was modelled with Gaussian Mixture Models trained on sample images. To reduce computational complexity, the 3D HSV colour distribution is projected onto the Hue-Saturation (HS), Hue-Value (HV), and Saturation-Value (SV) planes. Pixels within these predefined thresholds on these three planes were segmented as fire regions. Although their approach is originally applied to fire detection, the same principle can be adapted for QR code detection, as it can be utilised to recognise the colour of QR codes. However, a key limitation is the need for a large training dataset to capture colour variations accurately and the high computational cost during training and modelling.

In addition, [7] discussed similarity-based segmentation, which includes the k-means clustering technique and the region growing technique. K-means clustering technique groups pixels based on colour similarity by pushing each pixel value towards cluster centres and iteratively updating the mean of assigned pixels until convergence, as shown in Figure 2.2.1. However, k-means clustering depends heavily on the initial placement of cluster centres, which often requires post-processing to address noise and over-segmentation and is prone to slow convergence. These drawbacks can result in inconsistent segmentation and potential latency, making it unsuitable for real-time systems.

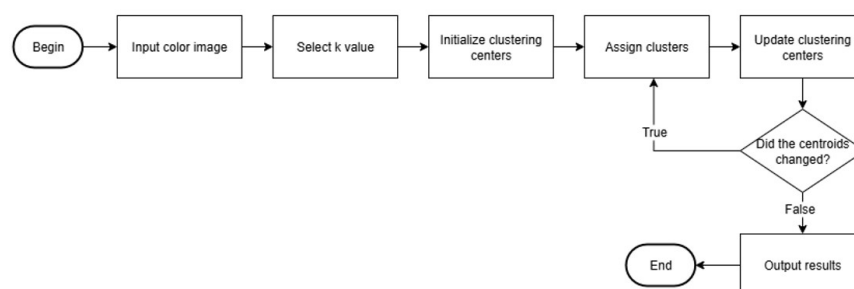


Figure 2.2.1: Process of K-means Clustering in [7]

CHAPTER 2

The region growing method groups neighbouring pixels into regions based on homogeneity criteria such as colour similarity and spatial proximity [7]. This method can generate smooth, contiguous segmentation masks, making it effective for isolating the target colour of the QR code. Based on Figure 2.2.2, the process begins from an initial seed point and expands by adding adjacent pixels that meet the predefined criteria. Despite this advantage, this method is highly sensitive to seed placement and homogeneity parameters, as an inappropriate seed can cause inaccurate segmentation.

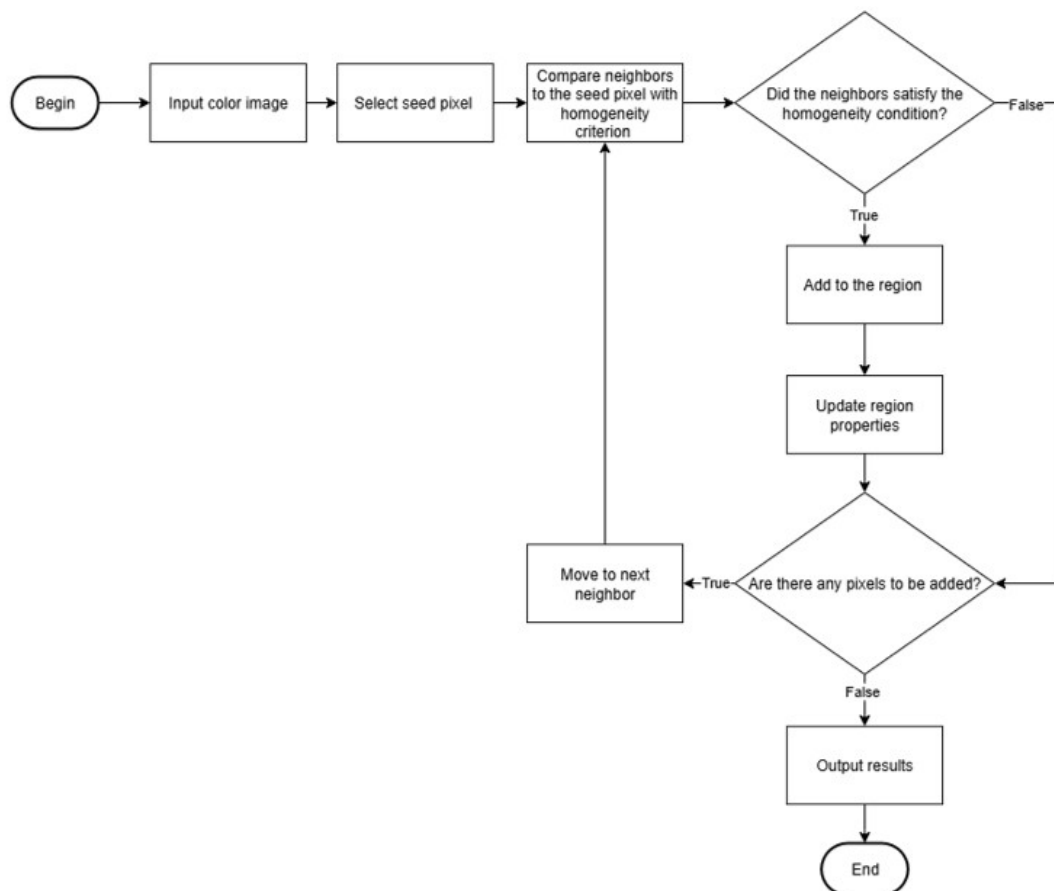


Figure 2.2.2: Process of Region Growing in [6]

2.3 Review of QR Code Detection Algorithms

The paper [8] developed an indoor localization and navigation system for mobile robots using QR codes. Although originally built for robots, their implementation can be adapted for a VI

navigation system. The authors placed the QR codes at fixed points in a grid-based interior area with each QR containing its location coordinates, with detection illustrated in Figure 2.3.1.

The camera captures each step of the robot's movement and transmits the images to a connected computer. The pictures are then transformed into gray-scale images to identify the shape of the QR code using feature extraction. The detected QR codes are decoded with ZBar and Open-Source Computer Vision (OpenCV), establishing the current position of the robot. A Dijkstra algorithm is applied to determine the shortest path to a target destination. Once a path has been determined, the robot would monitor its surroundings as it navigates, searching for QR codes continuously.

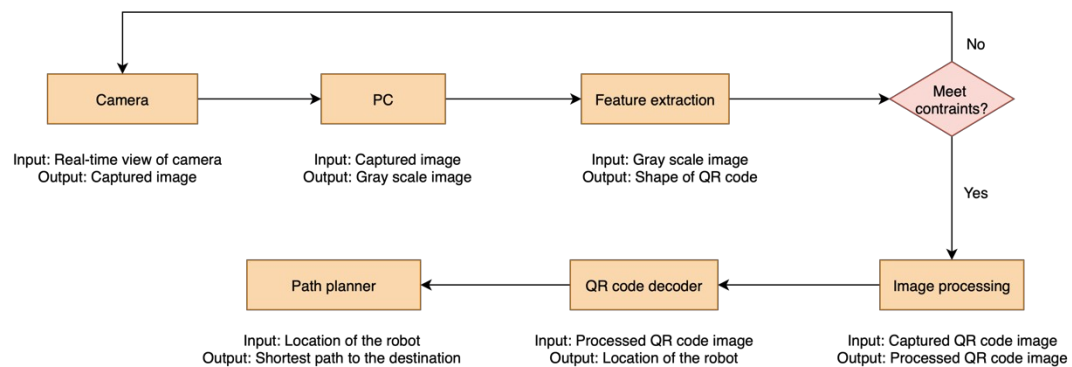


Figure 2.3.1 QR Code Detection Process of [8]

Huo et al. [9] suggested an artificial intelligence algorithm-based system for the recognition of QR image code. Their solutions consider issues faced by QR code recognition modules, such as background noise, poor lighting, and distortion. As illustrated in Figure 2.3.2 and Figure 2.3.3, the process begins with image capture, followed by grayscale transformation to reduce colour space and maximize contrast. Further image processing is done with an improved filtering algorithm to remove background noise. This is then followed by extracting the foreground QR patterns from the background using a threshold. If the image is distorted, it is corrected using a Backpropagation (BP) neural network-based distortion correction method. If there is a perspective distortion, a perspective inverse transformation is applied.

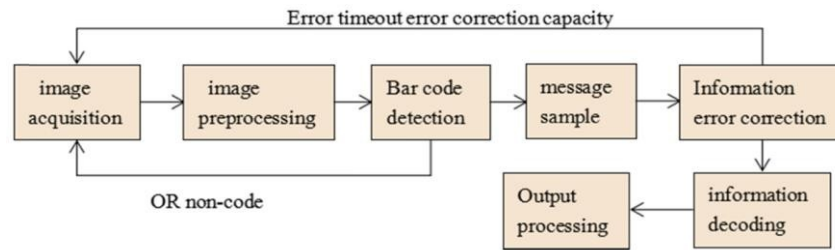


Figure 2.3.2 QR Code Recognition Process in [9]

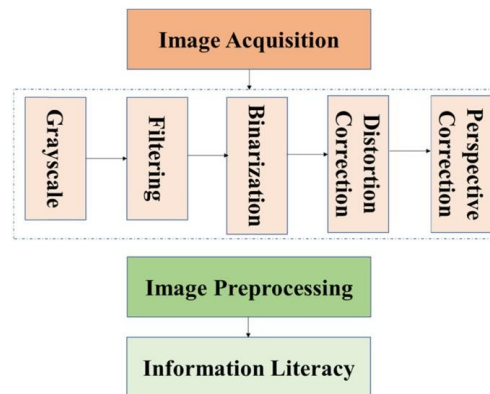


Figure 2.3.3 Image Preprocessing Process in [9]

2.4 Review of Technologies and Approaches in Indoor Navigation

Unlike traditional floor plan-based navigation solutions, [10] implemented a system that generates building architecture using Computer-Aided Design (CAD) models with a semantic mapping editor to manually update the map. The optimal path is computed with an A* algorithm, with an RGB-D camera capturing real-time depth data and images of the surroundings, a smart cane delivering haptic feedback, and a Time-Stamped Map (TSM) tracking obstacles to update navigation instructions. For user localization, [10] used the Singular Value Decomposition (SVD) algorithm to align the Area Description File (ADF) coordinate system with the semantic map, enabling accurate and consistent real-time localization within the environment.

Additionally, [10] used the TSM algorithm to create and update occupancy maps each frame, tracking obstacles dynamically as shown in Figure 2.4.1. However, the TSM system may face safety challenges in highly cluttered environments. To address this, [10] proposed the usage of

the Kalman Filter (KF) to estimate obstacle motion. KF will process dynamic obstacle data from TSM to estimate obstacle motion, allowing safer navigation in dynamic environments.

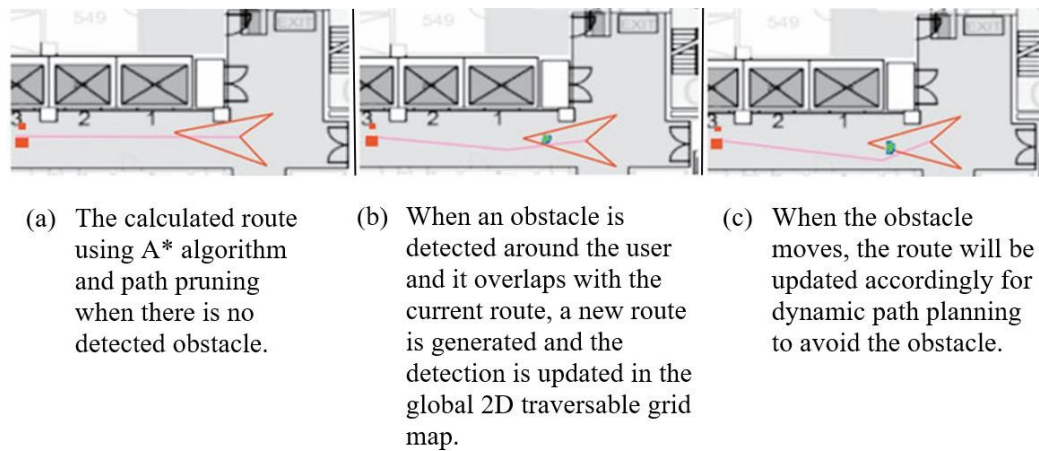


Figure 2.4.1: Navigation with Real-Time Obstacle Avoidance [10]

Additionally, a speech-to-text voice recognition module is integrated into a human-in-the-loop Human-Machine Interface (HMI), alongside cognitive waypoint path planning and smart cane support. This allows the system to provide general route guidance to the user, as illustrated in Figure 2.4.2.

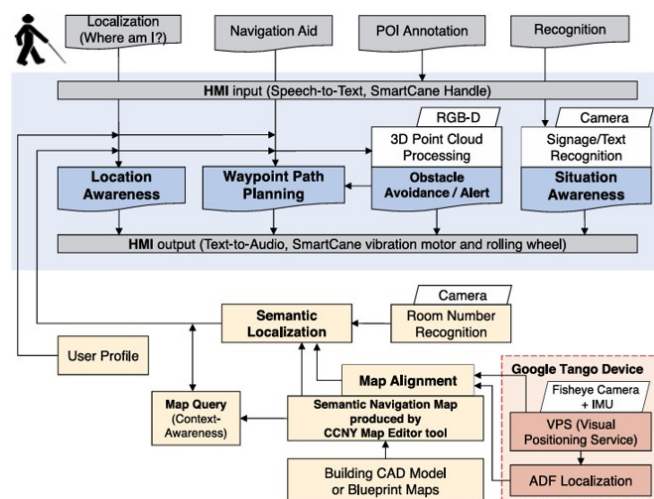


Figure 2.4.2: Navigation System Functional Block Diagram [10]

According to [11], a system was implemented that recognises specific scenes, providing situational awareness and offering navigation instructions as shown in Figure 2.4.3. For user localisation, [11] utilised the Speeded-Up Robust Features (SURF) algorithm, a vocabulary tree, a digital compass sensor, and a smartphone's built-in step-count sensor.

CHAPTER 2

Based on [11], the authors employed the SURF algorithm, paired with a vocabulary tree, for efficient image recognition. This approach detects and matches visual features from incoming images with a pre-indexed database to identify the user's location. With this, the user's real-time location can be determined, improving navigation. The limitations of the SURF algorithm and vocabulary tree are the need for high-quality training data to achieve fast and reliable real-time image recognition.

Furthermore, [11] proposed the use of smartphone-based sensors to enhance user real-time localisation as an alternative. A compass sensor to provide directional data and a step-count sensor to track the user's real-time movement. However, the compass sensor may be prone to magnetic interference from surrounding objects, and the step-count sensor may accumulate errors over long distances. To address this, [11] proposed integrating these sensors with the SURF algorithm and vocabulary tree to detect the user's current orientation and update routes in real time.

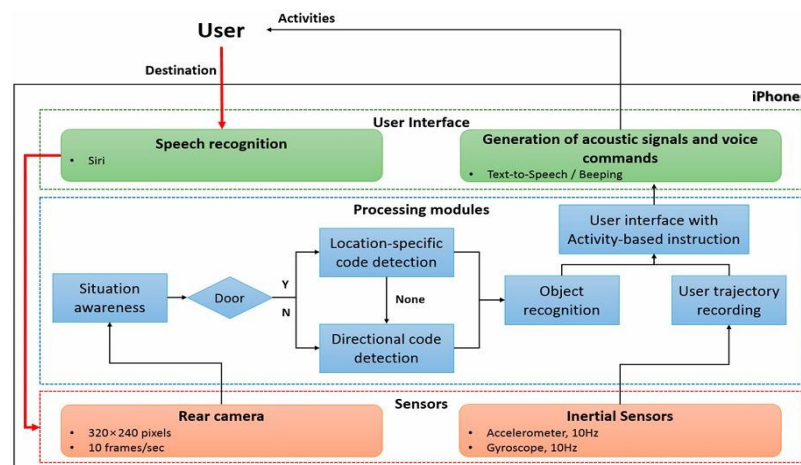


Figure 2.4.3: Architecture of The Navigation System [11]

2.5 Comparison of Methods

Done by: Every group member

2.5.1 Marker vs Marker-Less Approach

Table 2.5.1: Comparison Between Marker-Based and Markerless Approach

Feature/Aspect	Marker-Based Approach	Markerless Approach
Strength	Quick and efficient detection and localisation	No pre-installation of artificial markers
	More robust under varying light conditions	Adaptable to unfamiliar or dynamic environments
	Higher detection accuracy	No negative aesthetic impact on the environment
	Simpler detection process in controlled settings	High accuracy in position estimation
Weaknesses	Negative aesthetic impact due to physical marker placement	Require initial setup of feature/image databases or 3D models
	Effectiveness is limited by the marker installation	Sensitive to environmental changes
	Lighting conditions may affect QR code detection	High computational power and memory requirements
		High training data demand for AI-based methods
Shared Weakness	Lighting conditions, object occlusion, and dynamic scene elements may affect users' comfort due to the need to hold devices.	

2.5.2 Comparison of Colour Processing for QR Code Techniques

Table 2.5.2 : Comparison of Colour Processing Techniques

Techniques	Advantages	Disadvantages
Forest Fire Recognition Based on Feature Extraction from Multi-View Images [6]		
HSV Colour Space + Gaussian Mixture Model (GMM)	Provides an intuitive and effective representation of colours	Requires large dataset for training purpose

	Reduces computational complexity by projecting the 3D HSV distribution onto three 2D planes	Computational cost of training and modelling GMMs can be high.
Color Image Processing: Methods and Application (Chapter 5) [7]		
K-means clustering	Unsupervised clustering based on colour similarity	Performance depends on the initial clusters' centers
	Simple and easy to implement	May require post-processing to handle noise and over-segmentation
Region growing	Groups neighbouring pixels based on colour similarity and spatial proximity	Sensitive to the initial seed placement and parameter selection
	Seeded and unseeded variants allow flexible application.	Result can be wrong if seed or homogeneity criteria are poorly chosen.

2.5.3 Comparison of QR Code Detection Techniques

Table 2.5.3: Comparison of QR Code Detection Techniques

Techniques	Advantages	Disadvantages
Localization and Navigation Using QR Code in the Indoor Environment [8]		
QR Code Grid Map System	Enables accurate position localization in indoor environments	Static and environment dependent as system fails if QR codes are distorted.
Camera-based Image Capture	Uses a simple industrial camera with upward	Detection accuracy can be affected by insufficient

	orientation, suitable for fixed ceiling-mounted QR codes.	lighting, motion blur, or if the robot moves too fast.
QR Code Detection and Decoding (OpenCV and ZBar)	Fast and lightweight decoding pipeline	Performance may degrade under poor image quality or when QR codes are distorted.
Research on QR image code recognition system based on artificial intelligence algorithm [9]		
Improved Adaptive Median Filter	Improved accuracy in detecting noise points.	May increase the number of times the filter window has to expand.
BP Neural Network for Distortion Correction	Effectively corrects surface distortion.	Requires a large dataset of training samples.
OSTU algorithm for binarization	Simplifies the QR code into binary (black/white) for decoding	Not effective on images with complex backgrounds or severe noise.
Perspective Projection Inverse Transformation	Allows QR code to be properly interpreted even after severe geometric warping.	Not effective alone as it requires good distortion correction beforehand.

2.5.4 Comparison of Navigation Techniques

Table 2.5.4: Comparison of Navigation Approaches

Technique Used	Advantages	Disadvantages
Vision-Based Mobile Indoor Assistive Navigation Aid for Blind People [10]		
CAD Models + Semantic Map Editor	Supports path planning from building data.	Needs accurate CAD models and manual updates.

CHAPTER 2

A* Algorithm	Finds optimal path efficiently on grid maps.	High computation time in dynamic environments.
SVD for ADF Alignment	Improves localization accuracy.	Sensitive to localization drift if the environment changes.
Time-Stamped Map TSM	Detects obstacles in real time.	Struggle in cluttered environments.
Kalman Filter	Predicts obstacle motion.	May not match real-world behavior.
Cognitive Waypoint Path Planning	Provide step-by-step guidance for navigation.	Complex in dynamic layouts.
A Vision-Based Wayfinding System for Visually Impaired People Using Situation Awareness and Activity-Based Instructions [11]		
SURF algorithm	Fast, reliable for real-time image recognition.	Performance drops in cluttered areas.
Vocabulary Tree	Recognizes many scenes quickly.	Time-consuming to build and requires good training data.
Digital compass	Resolve direction ambiguity.	Prone to magnetic interference.
Built-in Step-Count Sensor	No external hardware and low cost.	May accumulate errors over long distances (drift issue).

CHAPTER 3

System Overview and Design

3.1 Design Overview

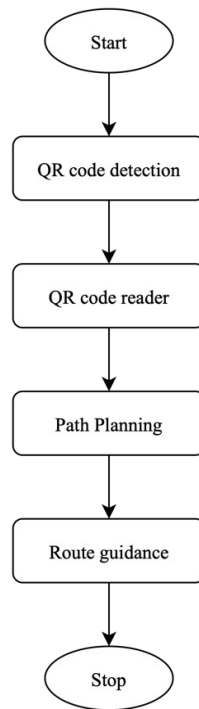


Figure 3.1.1.1: System Overview

The system comprises four major modules – QR code detection, QR code reader, path planning, and route guidance – along with an initiating action for colour and destination input. This system uses QR codes as an alternative to indoor positioning technologies such as Wi-Fi localisation, Bluetooth Low Energy (BLE) beacons, and RFID.

The process begins when the user selects a destination. Upon selection, the system initiates scanning for QR codes in the surrounding area. Within the QR code detection module, colour segmentation and QR code identification operate concurrently.

Once a QR code is detected, it is cropped and passed into the QR code reader module. The location data from the QR code is then used to compute the navigation path using the A* algorithm in the path planning module. Finally, the route guidance module delivers step-by-

step instructions via text-to-speech, guiding the user towards the next expected QR code, which serves as a navigation checkpoint.

3.2 System Design

3.2.1 QR Code Detection

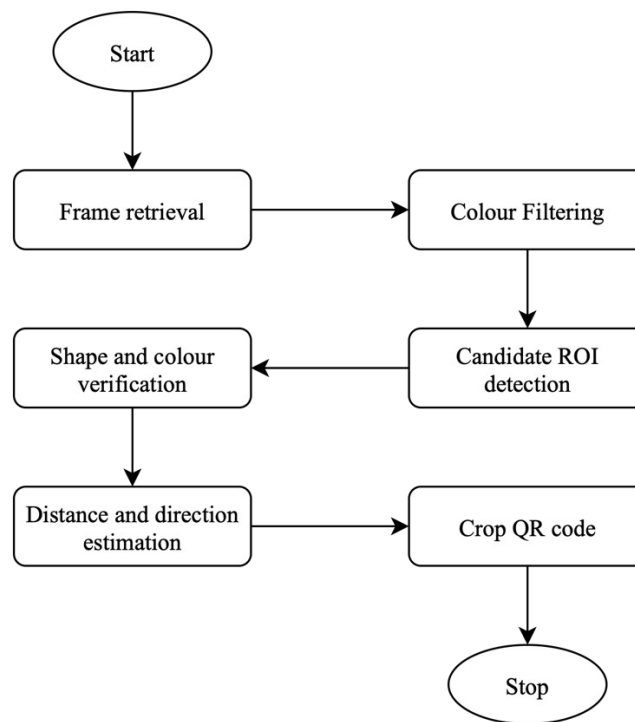


Figure 3.2.1.1: QR Code Detection Module

For this stage, the system focuses on identifying and processing coloured QR codes. If a QR code is detected, the system will provide step-by-step guidance to navigate the user to the most optimal position to read it. The process begins with frame retrieval, whereby a single image frame from the live video feed of the camera is captured. This frame acts as the raw input for the detection pipeline. By working frame-by-frame, the QR code detection pipeline can operate continuously while confirming the presence of a valid frame before processing, thus preventing empty or invalid inputs.

Once a frame is obtained, the system applies colour filtering to isolate regions that match the chosen colour. The acquired frame is first converted to the HSV colour space, providing a more

CHAPTER 3

reliable basis for colour segmentation compared to the RGB model. A binary mask is then generated by thresholding the HSV image within the target hue range.

The filtered mask is then passed to the candidate ROI detection stage. Here, contour detection is used to identify distinct areas of the coloured regions. Each contour undergoes validation checks such as minimum area and aspect ratio to identify shapes that are similar to the structures of QR codes. These selected contours are then converted to bounding rectangles, defining the candidate regions of interest (ROIs). By focusing on these ROIs, the system avoids scanning the entire frame, improving efficiency.

Each ROI is then processed using the QR code detection algorithm to confirm the presence of a valid QR pattern. If a QR code is found, it is passed to another layer of verification – shape and colour verification – which ensures that the detected region has a plausible geometry for a QR code and that its colour matches the intended target, thereby minimising false positives.

Once the region passes verification, the system proceeds to distance and direction estimation. Using the size of the QR code in the image and its position related to the frame's centre, the system estimates how far away the QR code is and whether it lies to the left, right, or directly ahead of the camera's current orientation. This spatial information is necessary to guide the user towards the QR code.

Finally, the verified QR code region is cropped from the original frame, producing a clean image that can be passed to the QR reader module for decoding.

3.2.2 QR Code Reader

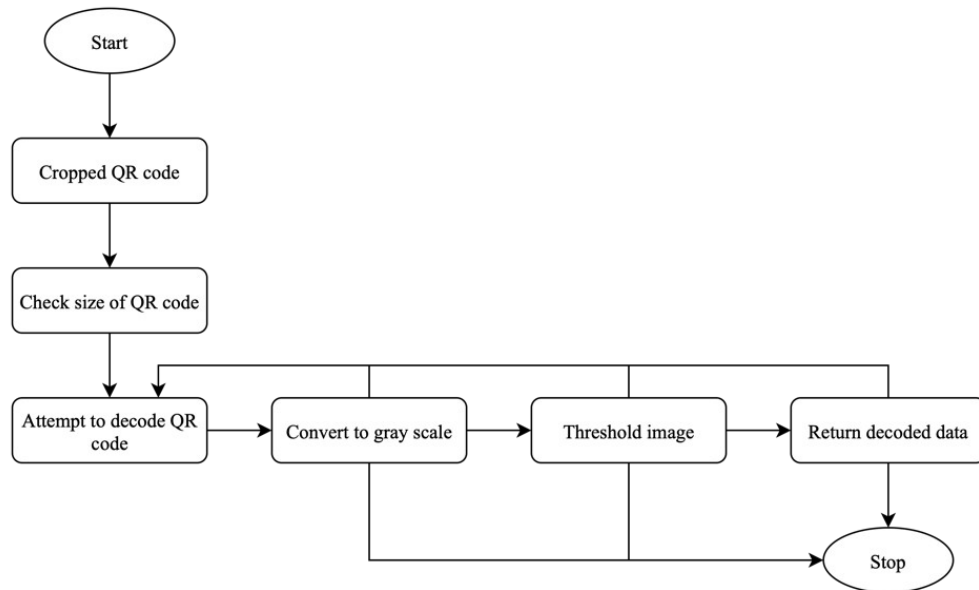


Figure 3.2.2.1: QR Code Reader Module

This subsystem is responsible for decoding the cropped QR code produced by the previous stage, the QR code detection module. It first checks the size of the QR code to ensure that the cropped image is large enough to contain sufficient detail for reliable recognition; if the image is below the minimum acceptable size, the process terminates early. This check serves as a safety net to handle edge cases.

If the size is adequate, the initial decoding attempt is made directly on the cropped QR code image in its original form. If this attempt is unsuccessful, the system converts the cropped image from colour to greyscale, leaving only the intensity value. This simplifies the data and can improve the contrast between the QR code's dark modules and light background.

If the decoding attempt is still unsuccessful, the module applies Otsu's thresholding. This converts the greyscale image into a binary format, separating the QR code from the background.

Once thresholding is completed, the module makes a final decoding attempt on the processed image. If successful, the decoded data – room IDs – are returned as the output of the module.

If unsuccessful, the module returns an empty string, signalling that the cropped image was unreadable. The progressive elevation of processing – from minimal to more aggressive only

when necessary – ensures that the QR Reader module operates efficiently in varying environmental conditions.

3.2.3 Path Planning

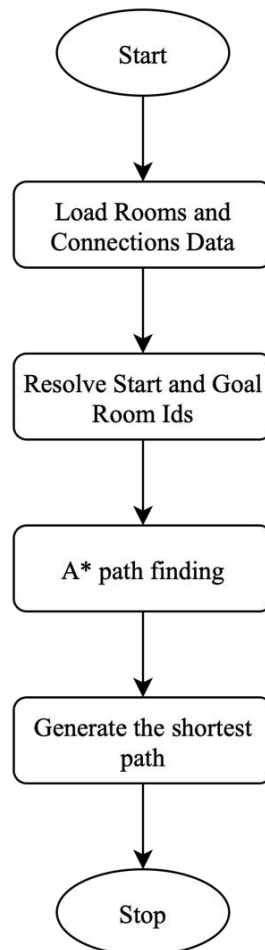


Figure 3.2.1.1: Path Planning Module

After determining the user's starting location, the system loads two text files, rooms.txt and connections.txt, which define all the rooms in the faculty and specify how they are connected. Together, these form a graph structure for the A* algorithm. The former file, rooms.txt, contains a list of all locations in the faculty, including their IDs, names, types such as classroom, laboratory, toilet, entrance, coordinates, and dimensions. Each entry represents a node in the navigation graph. The latter file, connections.txt, defines the links between these locations,

CHAPTER 3

specifying which nodes are connected and the type of connection. These entries form the edges of the graph used by the A* algorithm.

The A* algorithm operates on this graph to determine the shortest route from the current location to the selected destination. The A* algorithm formula is defined in (3.2.3.1), where $g(n)$ refers to the cost from the start node to a travelled distance, n , and $h(n)$ is the heuristic estimate of the cost from the latest travelled distance, n , to the goal node. Together, they provide the total estimated cost of the cheapest path through node n .

$$f(n) = g(n) + h(n) \quad (3.2.3.1)$$

The heuristic, $h(n)$, is calculated using the Euclidean distance between the destination and the current location, providing an estimate of proximity to the goal.

From the starting node, the algorithm repeatedly selects the node with the lowest $f(n)$ value, expands its neighbouring nodes from the connection list, and calculates the travel cost for each. If a shorter path to a neighbour is found, the neighbour's cost and parent reference are updated, and it is re-inserted into the open set. This process continues until the goal node is reached or no path remains.

Once the goal is found, the algorithm will produce the correct order of traversal by retracing its steps, generating a smooth sequence of node IDs – main entrance, junctions, or room codes.

This final path is then passed to the route guidance module, where it is converted into step-by-step navigation instructions for the user.

3.2.4 Route Guidance

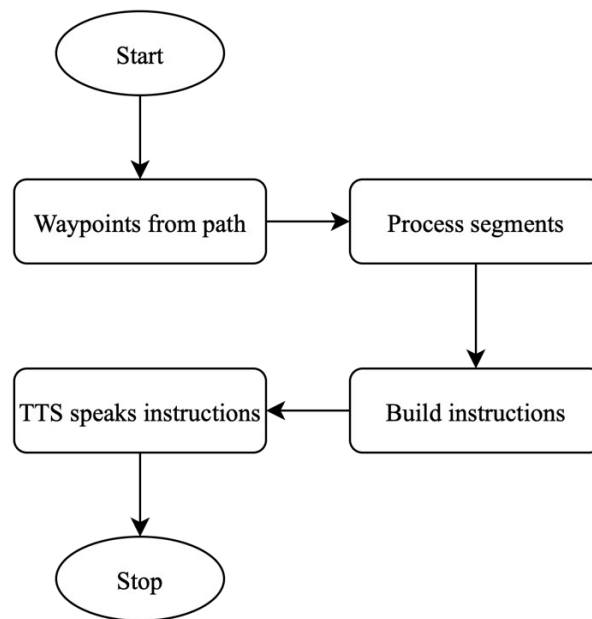


Figure 3.2.4.1: Route Guidance Module

Once the A* algorithm has computed the optimal route, the system transitions from path planning to instruction generation. At this stage, the path is represented as an ordered list of node IDs, each corresponding to a physical location defined in `rooms.txt`. This module converts the sequence of nodes into a set of spatial waypoints by retrieving the coordinates of each node and arranging them in the order provided by the path.

The resulting waypoints are then processed segment by segment, where each segment represents a connection between two consecutive waypoints. For each segment, the system calculates the physical distance in metres by applying the configured unit scale to the coordinate difference. In addition, an estimated number of steps based on the calculated distance and general average step length is generated, allowing the instructions to be tailored to a more intuitive, human experience. The system also determines the bearing of the segment and compares it to the bearing of the previous segment. Essentially, this comparison allows the generation of turning instructions, such as “continue straight”, “turn left”, “turn right”, or “make a U-turn”, depending on the degree of directional change.

CHAPTER 3

To provide the instructions with more context, the module inserts anchor points into the instructions. If a segment ends at a notable location, such as a classroom, an anchor phrase, “to N005”, will be added. This process repeats for each segment until the final waypoint is reached. The instruction list starts with a starting statement, such as “Starting at N005”, and a concluding statement, such as “Arrive at NG-001”.

These components – turn phrase, anchor phrase, and distance and step phrase – are concatenated into a clear, human-readable instruction. Once the instructions are created, they are read to the user with a text-to-speech feature, allowing a hands-free, accessible, and context-aware experience for VI users.

3.3 Implementation Issues and Challenges

One of the core technical challenges encountered in this project was maintaining high accuracy of QR code detection. Under poor lighting, the QR code detection module might lose its high accuracy, as reduced contrast and increased image noise made it harder for the camera to distinguish the code’s patterns, potentially leading to slower recognition or failed scans. Moreover, lighting shifts, glare, or excessive shadow may shift how the camera perceived a colour; a blue QR code might look purple under warm light, while under dim light, it might appear greyish. Furthermore, if a coloured QR code blends with the surrounding area, relying on colour filtering could make it harder for the detector to reconstruct the code. Additionally, the introduction of colour segmentation forced the need to find the optimal threshold value; if it is too strict, too much of the QR code might be cut, leaving it unreadable.

Another challenge arose from the implementation of the A* algorithm. Although A* algorithm computes the shortest path from one location to another, it does not take obstacles – students walking in hallways, animals, and moving objects in real time – into account. These obstacles might appear after a path was calculated, causing deviation and requiring the algorithm to execute again, increasing computational load and consequently slowing down navigation.

CHAPTER 3

Finally, designing a QR code detection system to be modular and maintainable posed its own challenges. First, the system designed must support future updates, such as changing algorithms or integrating new detection algorithms, without major rewrites of the rest of the application. Achieving this flexibility required thorough planning, careful separation of concerns, and well-defined interfaces between components. However, this approach might overcomplicate implementation, so it must be balanced with keeping the system lightweight and efficient for real-time use. In addition, any lag or mismatch between QR code detection and path computation could result in outdated or incorrect navigation instructions, exemplifying the importance of ensuring data flows smoothly between modules.

CHAPTER 4

System Implementation

4.1 Hardware Setup

Processing Unit

A laptop was used to execute the processes to aid the visually impaired (VI) in navigation. The built-in webcam was used to capture real-time video of the surrounding environment, which was then processed with colour masking, QR detection, and subsequently converted into navigation commands. As shown in Table 4.1.1,

Table 4.1.1 Specification of Processing Unit

Description	Specification
Model	Latitude 3410
Operating System	Linux Mint 22.1
Central Processing Unit (CPU)	Intel(R) Core (TM) i5-10210U CPU @ 1.60GHz
Graphical Processing Unit (GPU)	Intel UHD Graphics
Random-Access Memory (RAM)	16GB DDR4 2667 MHz
Storage	256GB SSD + 1TB HDD

4.2 Software Setup

4.2.1 Development Environment and Programming Language

The development of this project was done primarily in Visual Studio Code (VS Code), with its specifics shown in the table below (Table 4.2.1.1)

Table 4.2.1.1 Description of Visual Studio Code

Description	Specification
Version	1.104.0
OS	Linux Mint 22.1
Software dependencies and external libraries	Refer to Table 4.2.2.1
C++ version	GNU GCC 13.3.0
Purpose	For development and testing

CHAPTER 4

C++ was chosen as the primary programming language for development due to its versatility, with support for both procedural and object-oriented paradigms; its readability, as it is statically typed, reduces the susceptibility to errors; and its more granular control through its pointer system. The primary library used in this project was OpenCV for its vast computer vision functionalities and its support for building simple graphical user interfaces to display and interact with processed images.

4.2.2 Software Specification

Table 4.2.2.1 Software Specification

Libraries and Dependencies	Version	Description
Image processing For image manipulation, conversion, and analysis		
OpenCV	4.11.0.86	A library for computer vision tasks
ZBar	0.23.93-build3	A library to scan QR codes from image or video streams
GUI		
OpenCV	4.11.0.86	Create partitions to show different processing stages and text overlay to show the scanned QR and suggested direction
Data handling To store and handle the data structure of the layout of the faculty and the navigation suggestions between locations		
.txt	standard	Main format to store the different rooms within the faculty and the connections between them
map	GNU libstdc++ 13.3.0	Stores key-value pairs in sorted order
unordered_map		Stores key-value pairs in a hash table
unordered_set		Stores a collection of unique elements
vector		Implements a dynamic array
set		Stores unique elements
queue		Provides a FIFO container
optional		Represents values that may or may not be present
limits		Defines numeric limits for data type validation and boundary checks
Data Analysis Algorithms and computations on data		
algorithm		For searching, sorting, and manipulating data

cmath	GNU libstdc++ 13.3.0	Mathematical functions for numerical computations
utility		Supplies utility components for generic programming
File System Interaction Reading and writing files		
fstream	GNU libstdc++ 13.3.0	Enables reading from and writing to files
sstream		Allows string-based stream operations for parsing and formatting
iostream		Provides standard input/output stream functionality
Utility General-purpose library		
string	GNU libstdc++ 13.3.0	Manages and manipulates sequences of characters
chrono		Measures and manipulates time intervals

4.2.3 Input Source Setup

This setup is designed to accommodate a variety of image sources – local folders and a webcam.

Table 4.2.3.1 Input Sources Description

Input Source	Configuration
Local Folder	Connect the folder directly in the application
Webcam	Connect the webcam directly to the application

4.2.4 File Operation and Data Storage Setup

The project's file structure is organised into three main categories: core files, module files, and utility files. The **core** folder contains the main application framework and the user interface controller, which coordinate interactions between different modules – including the QR reader, QR detector, and navigation components. The **modules** folder contains self-contained, single-purpose components that implement specific functionalities, enabling modular development and easier maintenance. Finally, the **utility** folder contains supporting classes and configuration files that define system presets and essential parameters. This folder is kept read-only during

CHAPTER 4

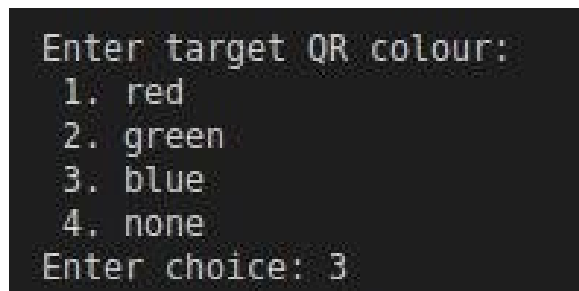
execution to ensure that all critical static files, such as text files describing the faculty's room layout and interconnections, remain intact and accessible regardless of the system configuration or runtime state.

4.3 System Operation

This section describes the system's operation aspects of the graphical user interface (GUI). It details what each component in the graphical user interface means and why they are important. The system is made with the assumption that the user can click and type.

4.3.1 Input Section

On startup, the user will be prompted to insert a target colour (Figure 4.3.1.1). If an invalid choice is inserted, the user will be defaulted to no colour. Once inserted, the system will reiterate the user's choice.



```
Enter target QR colour:
1. red
2. green
3. blue
4. none
Enter choice: 3
```

Figure 4.3.1.1: Inserting Target Colour

After that, the system will prompt the user to insert a destination. Once the user has inserted the destination, the system will reiterate the user's choice (Figure 4.3.1.2).



```
Enter destination room ID (e.g., N008): ng-001
```

Figure 4.3.1.2: Inserting Destination

4.3.2 System Display

The user interface (UI) of the system comprises four partitions – the camera feed, masking, and the cropped-out detected QR code and text information – as shown in Figure 4.3.2.1. Although visually impaired (VI) individuals may not be able to see these partitions, they provide crucial

CHAPTER 4

details about the data fed into the system. The colour masking partition highlights all the objects that fall within the range of the selected colour, as depicted in Figure 4.3.2.1. Despite its simplicity, this partition denotes how lighting shifts can affect how colour is perceived by the system.



Figure 4.3.2.1: System Partitions

When a QR code is found to be within range of the camera, the system will draw a bounding box on the QR code and prompt the user to move to an optimal position (Figure 3.2.2.2). The detected QR code will be displayed beneath the masking partition, allowing us to observe whether QR codes can still be detected when viewed at the edges of limits – angle, distance, and tilts.

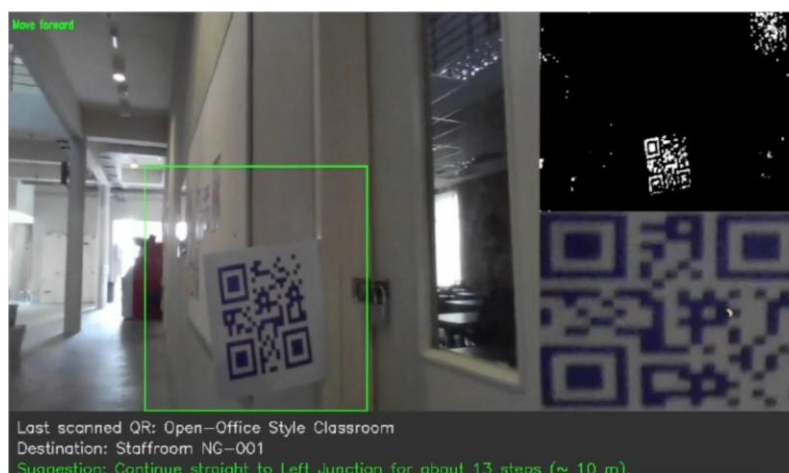
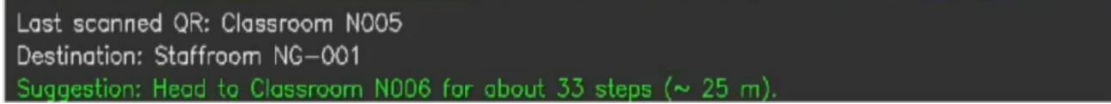


Figure 4.3.2.2: Cropped QR Code Partition

CHAPTER 4

Finally, the text partition details the last scanned location of the user, the desired destination, and the navigation suggestion to the next QR code (Figure 4.3.2.3). This partition is necessary to ensure that the right information is being received and conveyed to the user.



Last scanned QR: Classroom N005
Destination: Staffroom NG-001
Suggestion: Head to Classroom N006 for about 33 steps (~ 25 m).

Figure 4.3.2.3: Text Information Partition

4.4 Concluding Remark

This chapter has outlined the technical implementation of the system, covering hardware and software tools, software architecture, and system operation, demonstrating a balance of real-world practicality and user experience. Key features include the extensibility of the system with modular design and system architecture, allowing additional functionalities to be incorporated without major structural changes. Moreover, the system presents relevant outputs to the user, such as bounding boxes and cropped QR codes, to support navigation and feedback.

While there are challenges during implementation, notably around determining ROI, these were addressed through continuous, iterative feedback and refinement.

In conclusion, this project provides a foundation for current and future development regarding coloured QR code-based indoor navigation systems, tailored for real-world use cases.

CHAPTER 5

System Evaluation and Discussion

5.1 System Testing and Performance Metrics

To evaluate the effectiveness and readiness of the developed system, a series of tests was conducted on each module. System testing is a critical phase that validates the complete, integrated system against its requirements. It examines the overall workflow and the interactions between modules to ensure they perform as expected under real-world conditions.

The testing procedure included both functional and non-functional assessments. Functional testing verified that each module produced the intended outputs for valid inputs, while non-functional testing evaluated system performance in terms of accuracy and robustness under various operational environments. Through this comprehensive testing, the system evaluation validated the system's functionality and identified areas that may require improvement.

5.1.1 QR Code Detection

The objective of this evaluation was to assess the system's reliability and accuracy in detecting the coloured QR codes across different environmental settings and usage scenarios. The system testing was carried out by defining a set of performance metrics to act as the benchmarks for the evaluation. Detection success rate was used to measure the percentage of QR codes correctly detected out of total attempts, as defined in (5.1.1.1). To evaluate the robustness of the module, tests were conducted under varying conditions such as different lighting levels, distances, QR code sizes, and patterns. This approach helped to ensure the stability and consistency of the module's performance under different scenarios. Table 5.1.1.1 defines the robustness variable applied for the testing phase.

$$\text{Detection success rate} = \frac{\text{Total correctly detected QR Code}}{\text{Total attempts of scanning the QR Code}} \times 100\% \quad (5.1.1.1.1)$$

Table 5.1.1.1: Robustness Test Variables for the QR Code Detection Module

Variable	Condition	Purpose
Lighting conditions	Low light, Bright	To test the performance under varying lighting conditions
Distance	30cm, 60cm	To test detection distance
QR code area	Small (100cm ²), Large (256cm ²)	To test which size can be detected most easily
QR code pattern	Blue module + White background, White module + Blue background, Blue module + Black background, Black module + Blue background	To determine which pattern that can be detected most easily

Table 5.1.1.2 summarises the selected metrics and their measurement methods, along with the target goals defined for the testing phase.

Table 5.1.1.2: Performance Metrics Summary for the QR Code Detection Module

Metrics	Measurement	Goals
Detection success rate	The detection success rate will be calculated as the percentage of QR codes correctly detected out of 30 test attempts.	The system should achieve a high detection success rate under all tested conditions.
Robustness	The system will be tested under different conditions.	The system should maintain stable and consistent detection performance under all tested conditions.

5.1.2 QR Code Reader

The evaluation for the QR code reader module was conducted using the QR codes detected during one of the real-world navigation tests. In total, the system captured 37 QR codes during navigation, a high count resulting from continuous scanning of QR codes. For evaluation purposes, all detected QR codes were used, regardless of repetition, as illustrated in Figure 5.1.2.1.



Figure 5.1.2.1: QR Code Detected Images

Amongst the 37 images, 8 of them were captured in a bright environment as shown in Figure 5.1.2.2, and 23 of them were captured in darker lighting as shown in Figure 5.1.2.3. Additionally, Figure 5.1.2.4 shows 6 QR code images that were captured with uneven brightness.

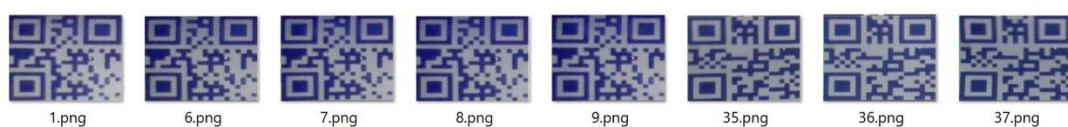


Figure 5.1.2.2: QR Code Images Under Bright Environment

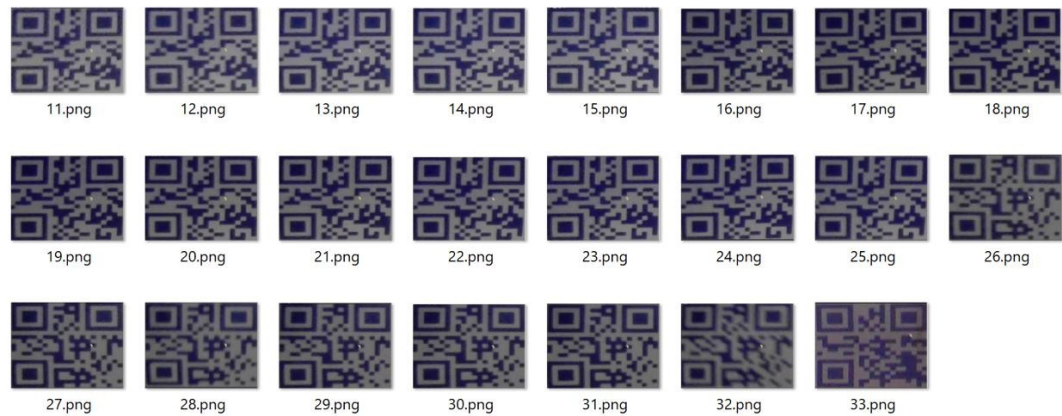


Figure 5.1.2.3: QR Code Images Under Low Light Environment

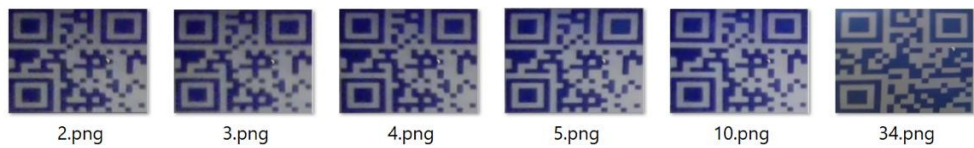


Figure 5.1.2.4: QR Code Images with Uneven Brightness

The module will be evaluated as shown in the table below.

Table 5.1.2.1: QR Code Reader Module Evaluation Details

Metrics	Measurement	Goal
The module shall achieve a decoding accuracy of at least 90% for all QR codes captured in varying lighting conditions.	Accuracy will be measured by determining if the module is able to decode QR code under various lighting conditions	Continuous improvement to ensure that accuracy exceeds at least 95%

Additionally, the QR code images were presented in three partitions, comprising the original, greyscale, and binarized versions. This was to visualise the transformation process used in the decoding pipeline. If decoding was successful in any of the prior stages, the remaining partitions remained blank. Displaying all three stages was necessary to demonstrate how the module progressively refines the input to maximise decoding accuracy. However, the evaluation did not test QR codes that were angled or skewed, as the detected QR codes had already performed perspective correction, creating a flat image.

CHAPTER 5

The decoding results were displayed beneath the three image partitions in the format “QR decoded: <content>” or “QR decoded: None” to indicate whether the QR code was successfully decoded. Moreover, the evaluation specified the stage in which the QR code was decoded, for instance, “QR decoded on original image” or “QR decoded on grayscale image”. This will provide traceability between each stage and the outcome, allowing performance to be assessed for each image type.

The total number of QR codes successfully decoded from each stage was calculated, along with the total number of failed decoding trials. The success rate is calculated as follows (5.1.2.1).

$$\text{Success rate} = \frac{\text{Total QR Codes Decoded}}{\text{Total QR Code Tested}} \times 100\% \quad (5.1.2.1)$$

5.1.3 Path Planning

In this section, the path planning module, which uses the A* algorithm, was evaluated against path correctness (optimality), validity, and efficiency using a faculty indoor map derived from rooms.txt (room IDs, names, types, and coordinates) and connections.txt (undirected links).

Path correctness and optimality were validated by comparing the A* algorithm’s path length to those estimated by Dijkstra’s algorithm. [12] claims that when the heuristic is zero, the A* algorithm operates similarly to Dijkstra, and the shortest routes from the A* algorithm are the same as Dijkstra’s. Therefore, Dijkstra’s path length was used as the ground truth for evaluating the A* algorithm, and Dijkstra’s results were adopted as the reference for validating path correctness and optimality.

After the Dijkstra and the A* algorithm’s path lengths were computed, the optimality ratio defined in (5.1.3.1) was evaluated to quantify similarity to Dijkstra. The optimality gap defined in (5.1.3.2) shows the absolute difference in metres. Aggregate statistics, including the mean, median, standard deviation, minimum, maximum of optimality ratio, and the percentage within a small tolerance, were also reported. The tolerance criterion defined in (5.1.3.3) accepts a result as optimal when the A* algorithm path length differs from Dijkstra’s by at most 0.1%.

$$\text{Optimality ratio, } \eta = \frac{\text{Path length of A* for a start to goal pair}}{\text{Path length of Dijkstra for a start to goal pair}} \quad (5.1.3.1)$$

$$\text{Optimality gap} = \text{Path length of A*} - \text{Path length of Dijkstra} \quad (5.1.3.2)$$

$$\text{Tolerance Criterion} = |\eta - 1| \times 10^{-3} \quad (5.1.3.3)$$

Validity was checked by counting the illegal edges (any consecutive pair in the reported path that is not an edge in the graph) and by flagging loop presence. A success rate defined in (5.1.3.4) indicates how many valid paths were expected to be found. A connectivity rate defined in (5.1.3.5) was measured to confirm that the map itself permits a path between the start and goal, and it was determined by checking whether Dijkstra returns a finite path length.

$$\text{Success rate} = \frac{\text{Valid path found by A*}}{\text{Total path}} \times 100\% \quad (5.1.3.4)$$

$$\text{Connectivity rate} = \frac{\text{Finite path length returned by Dijkstra}}{\text{Total path}} \times 100\% \quad (5.1.3.5)$$

5.1.4 Route Guidance Testing

For this evaluation, emphasis was placed on turn handling, since correctness at junctions is the most safety-critical aspect for visually impaired users. Table 5.1.4.1 and Table 5.1.4.2 show the test cases for turning handling.

Table 5.1.4.1: Turn Right Navigation Instruction Test

Turn Right Instructions			
Test ID/Types of Inputs	Start	Goal	Expected Output
ITC-001	N003	ENTRANCE_MAIN	Turn right
ITC-002	N011	JUNC_MID	Turn right
ITC-003	N008	N007	Turn right
ITC-004	NG-043	STAIR_L	Turn right
ITC-005	NGT2	ENTRANCE_MAIN	Turn right
ITC-006	N004	JUNC_MID	Turn right
ITC-007	NG-036	NG-028	Turn right

Table 5.1.4.2: Turn Left Navigation Instruction Test

Turn Left Instructions			
Test ID/Types of Inputs	Start	Goal	Expected Output
ITJ-001	N001	NGT3	Turn left
ITJ-002	N007	N008	Turn left
ITJ-003	N010	JUNC_RIGHT	Turn left
ITJ-004	JUNC_LEFT	N010	Turn left
ITJ-005	N012	N001	Turn left
ITJ-006	NG-011	NG-027	Turn left
ITJ-007	NGT1	N001	Turn left

5.1.5 Input Validation Testing

This section of the system evaluation assessed the robustness of the system in handling multiple types of input, including user inputs and the room ID stored in the scanned QR code.

Table 5.1.5.1: QR Colour Input Test

QR Colour Input		
Test ID/Types of Inputs	Inputted Data	Expected Output
IT-001 Numeric	1	You chose colour: red
IT-002 Numeric	2	You chose colour: green
IT-003 Numeric	3	You chose colour: blue
IT-004 Numeric	4	You chose colour: none
IT-005 Numeric	Any numerical value outside of [1,2,3,4]	Invalid choice. Please try again.
IT-006 Text	Enter a character or a string of character	Invalid input. Please enter a number

Table 5.1.5.2: Destination Room ID Input Test

Destination room ID		
Types of Inputs	Inputted Data	Expected Output
IT-007 Text	Small cases	You chose destination: [inserted destination]
IT-008 Text	Capital cases	You chose destination: [inserted destination]

IT-009 Text	Title cases	You chose destination: [inserted destination]
IT-010 Text	Unlisted room ID	Destination not found. Please try again.
IT-011 Numeric	Any numbers	Destination not found. Please try again.

Table 5.1.5.3: Scanned QR Code Input Test

Scanned QR code		
Types of Inputs	Inputted Data	Expected Output
IT-012 Text	Small cases	Last scanned QR: [Room ID] Suggestion: [Instruction]
IT-013 Text	Capital cases	Last scanned QR: [Room ID] Suggestion: [Instruction]
IT-014 Text	Title cases	Last scanned QR: [Room ID] Suggestion: [Instruction]
IT-015 Text	Valid room ID	Last scanned QR: [Room ID] Suggestion: [Instruction]
IT-016 Text	Invalid room ID	Last scanned QR: [decoded content] (unknown) Suggestion: No path found from [decoded content] to [destination]
IT-017 Numeric	Any numbers	Last scanned QR: [decoded content] (unknown) Suggestion: No path found from [decoded content] to [destination]

5.2 Testing Setup and Result

5.2.1 QR Code Detection

The testing for the QR code module was carried out at the ground floor of the FICT building in UTAR. A laptop camera with a resolution of 720p was utilised to capture and detect the QR codes. Based on Figure 5.2.1.1, the QR codes were attached on the glass on the doors. However, the preliminary trials revealed that detection was difficult when users walked perpendicularly to the doors, limiting the scanning angles. Addressing this issue, the QR codes were redesigned into a triangular prism structure, as shown in Figure 5.2.1.2. They were attached to the door

CHAPTER 5

panel as shown in Figure 5.2.1.3. Both square sides of the prism was printed with a QR code to allow scanning from multiple directions and improve the detection reliability.



Figure 5.2.1.1: QR Code Attached on Door Glass



Figure 5.2.1.2: QR Code with Triangle Prism Structure



Figure 5.2.1.3: QR Code Attached on Door Panel

For each test scenario, the QR codes were scanned three times to ensure reliable and consistent measurements. The laptop camera was positioned at two different distances from the QR codes (30 cm and 60 cm) and tested under two lighting conditions (low light and bright light). Several QR code patterns and sizes were tested under the same conditions to determine the configuration with the highest detection accuracy. The results of these tests are presented in Table 5.2.1.1.

Table 5.2.1.1: Detection Performance Under Different Conditions

QR Code Patterns	QR Code Area (cm ²)	Lighting Conditions	Detection Success Rate under different Detection Range (cm)	
			30	60
Blue module + White background	100	Low light	0%	0%
	100	Bright	0%	0%
	256	Low light	66.67%	0%
	256	Bright	66.67%	0%
White module + Blue background	100	Low light	0%	0%
	100	Bright	0%	0%
	256	Low light	43.33%	0%
	256	Bright	46.67%	0%
	100	Low light	0%	0%

Blue module + black background	100	Bright	0%	0%
	256	Low light	50%	0%
	256	Bright	53.33%	0%
Black module + Blue background	100	Low light	0%	0%
	100	Bright	0%	0%
	256	Low light	63.33%	0%
	256	Bright	66.67%	0%

As shown in Table 5.2.1.1, the system was able to detect a QR code with an area of 256 cm² at 30 cm, irrespective of the QR code pattern and lighting conditions. In general, larger QR codes had higher detection success rate compared to smaller QR codes. Among these configurations, the coloured module with white background and black module with blue background achieved the two highest detection success rates across all different conditions. Although the performance of both configurations was close, the coloured module with white background configuration provided higher successful detection rate and stability. Therefore, the coloured module with white background was chosen for the demonstration. The Figure 5.2.1.4 shows the real-world testing setup and results for this evaluation.

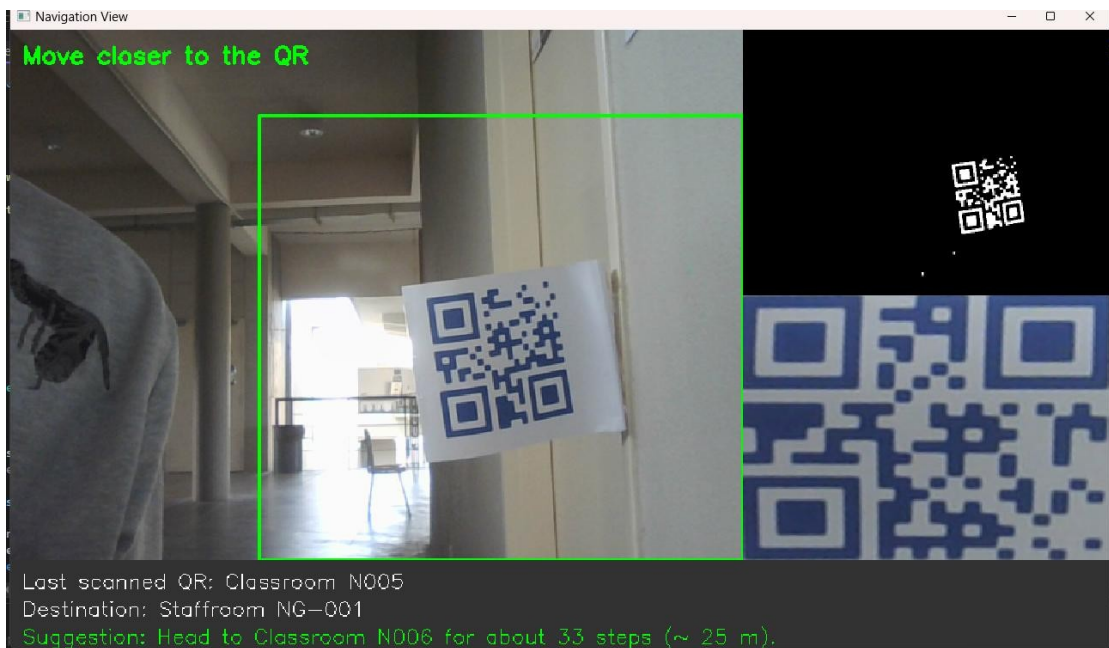


Figure 5.2.1.4: Example of Real-world Testing and Result

CHAPTER 5

Moreover, the system was also tested with additional QR code colours, such as green and red. The results indicated that detection for green-coloured QR code was less stable in comparison to the other two colours, underscoring the need to improve detection consistency. In addition, it was also found that the print quality of the QR codes affected detection performance. As illustrated in Figure 5.2.1.5, two QR codes with different printing qualities were compared, where the left QR code represented the low-quality print, while the right QR code represented the high-quality print. During detection, the system detected the left QR code as a pinkish colour, while the right QR code appeared as a distinct red colour. Therefore, high-quality printing is necessary to ensure a more reliable and accurate detection.

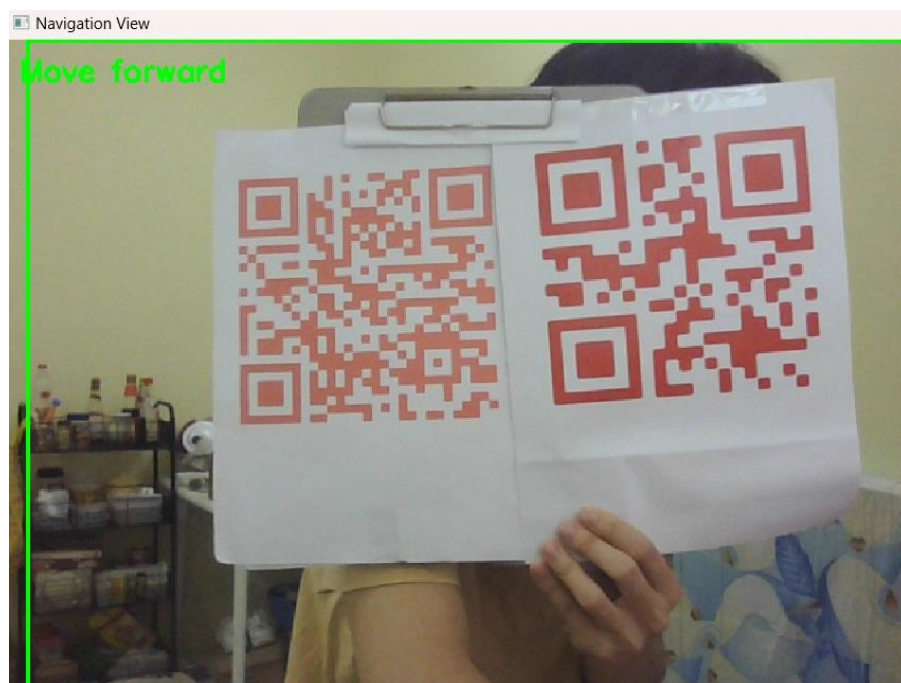


Figure 5.2.1.4: Comparison of QR Codes Between High-quality and Low-quality Print

5.2.2 QR Code Reader

Out of the 37 QR images, 8 of them, 21.62%, were successfully decoded from their original format, 7 images, 18.92%, were decoded once grayscale had been applied, and 16 of them, 43.24%, were only decoded when binarized, as shown in Figure 5.2.2.1. However, there were 6 images, 16.22%, that failed to be decoded even after all the modifications had been applied. This shows that the progressive image preprocessing steps are effective in increasing the decoding success rate. The overall success rate of the module was 83.78, as shown in Figure 5.2.2.2.

```

QRs decoded from original image : 8 (21.62%)
QRs decoded from gray image    : 7 (18.92%)
QRs decoded from binarized image : 16 (43.24%)
QRs failed to decode           : 6 (16.22%)
Success rate                    : 83.78%

```

Figure 5.2.2.1: Total QRs Failed and Successfully Decoded from Original, Gray, and Binarized

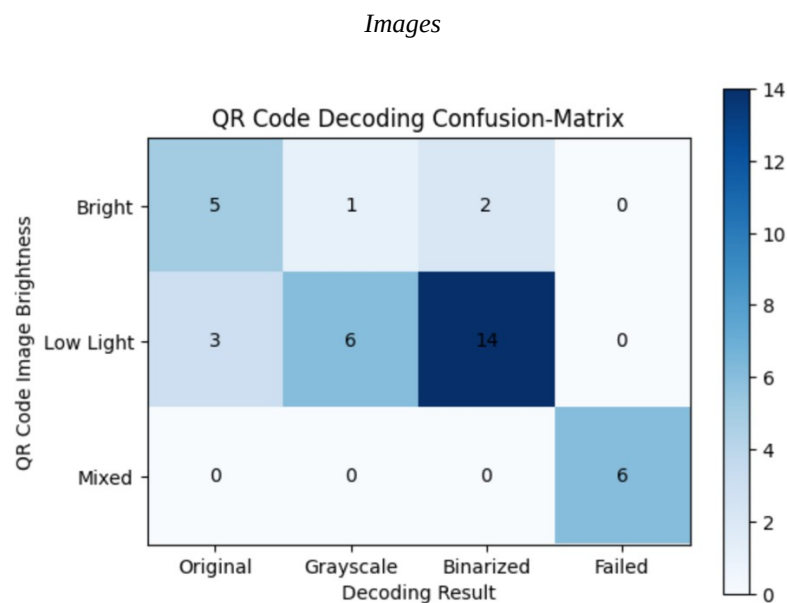


Figure 5.2.2.2 Confusion Matrix of QR Code Decoding Results

Analysing the attributes of each image along with preprocessing used, if any, can provide information regarding the best environment for a QR code to optimise resource usage and improve efficiency. Figure 5.2.2.3 shows an example of a QR code that was successfully

decoded without any preprocessing; the QR code captured was very clear and bright, allowing the shapes and patterns of the QR code to be easily recognised by the module.

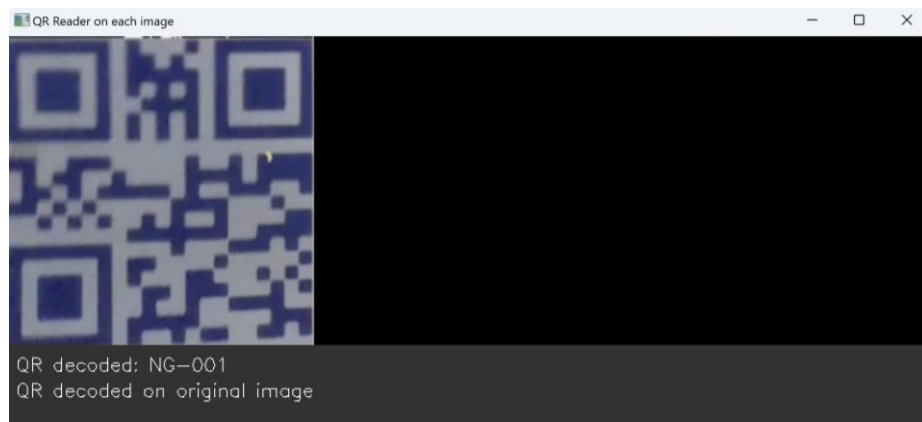


Figure 5.2.2.3: Example of QR Code Decoded on Original Image

QR codes that required conversion to grayscale were often slightly blurred and captured in a dimly lit environment. This is because dark environments reduce contrast, making it harder for the reader to detect the shapes and patterns of the QR code. Shifting the format to grayscale creates a clearer contrast between the patterns of the QR code and the background. Figure 5.2.2.4 shows a successful decoding after a QR code was converted to grayscale format.

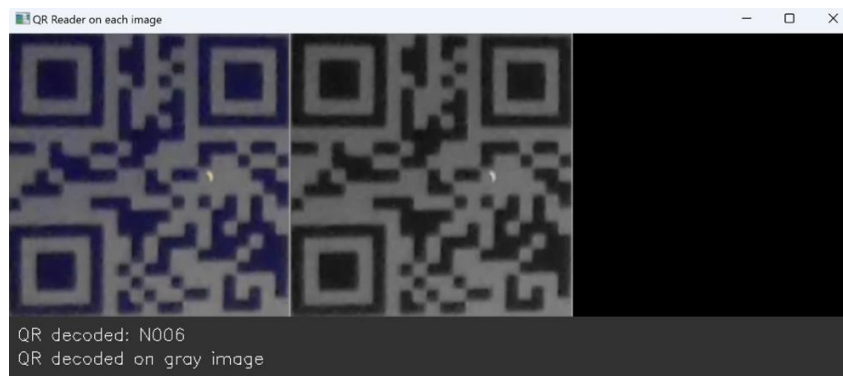


Figure 5.2.2.4: Example of QR Code Decoded on Gray Image

Finally, binarization or thresholding is useful for QR codes that are blurred. Binarizing QR code images will sharpen the blurred borders, making it easier to detect the QR code shape as shown in Figure 5.2.2.5.



Figure 5.2.2.5: Example of QR Code Decoded on Binarized Image

According to the tests conducted, QR code images with uneven brightness were more likely to fail. Since the module depends on contrast, uneven lighting may confuse the reader. Moreover, uneven brightness causes thresholding to fail as darker areas are converted to black, breaking the QR code's patterns, as shown in Figure 5.2.2.6.



Figure 5.2.2.6: Example of QR Code Failed to Decode

Thus, the module is effective in both brightly lit and dimly environments with its grayscale and thresholding feature, but it may fail in environments with uneven lighting, as it may alter how the QR code is presented to the system.

5.2.3 Path Planning

Both algorithms, the A* algorithm and the Dijkstra's algorithm, were used to generate paths and stored for comparison. The file contained test routes used to check whether the A* produced valid, optimal paths compared to the Dijkstra's algorithm. Its columns were TestID, Start, Goal, Success, PathNodes, LenAstar_m, LenDij_m, OptRatio, Expansions, Time_ms, IllegalEdges, and HasLoop.

CHAPTER 5

Descriptive analysis and visualization of performance metrics, including the success rate, connectivity rate, optimality ratio, optimality gap, expansions, and runtime insights, were conducted. Based on Figure 5.2.3.1, a total of 6162 routes were evaluated. A success rate of 100% was obtained with no illegal edges and no loops observed. This indicated the indoor graph is well structured and each node was fully connected.

```
Total routes: 6162
Success rate (found path): 100.0 %
Valid route rate (no illegal edges/loops): 100.0 %
Connected routes (finite Dijkstra): 100.0 %
```

Figure 5.2.3.1: Overview of The Route Sets

Figure 5.2.3.2 reports the optimality statistics, indicating that the optimality ratio, along with its mean, median, minimum, and maximum values, was consistently 1.000. Furthermore, both the optimality gap and the standard deviation of the optimality ratio were 0.000. These values satisfied the tolerance criterion and was accepted as optimal. The optimality gap (0.000) indicated that the A* algorithm produced path lengths identical to Dijkstra's shortest paths across all evaluated routes, confirming the planner's correctness and optimality on this map.

```
--- Optimality (A* vs Dijkstra) ---
Count: 6162
% within ±0.001 (tolerance) : 100.0 %
OptRatio: 1.0
OptGap (m): 0.0
Mean OptRatio: 1.0
Median OptRatio: 1.0
Std OptRatio: 0.0
Min/Max OptRatio: (Min) 1.0 (Max) 1.0
```

Figure 5.2.3.2: Optimality Statistics on Route Sets

Figure 5.2.3.3 shows the efficiency results, whereby the search required a median of 27 node expansions with 90% of routes solved in ≤ 56 expansions. The runtime was 1.505 ms^{-1} as the median and $\leq 3.747 \text{ ms}^{-1}$ for 90% of routes. This indicated that most queries were resolved with modest search effort and low latency.

```

--- Efficiency ---
Median expansions: 27.0
p90 expansions: 56.0
Median time (ms): 1.505
p90 time (ms): 3.7469000000000006

```

Figure 5.2.3.3: Efficiency Results on Route Sets

A visualisation of the evaluation on the route sets is presented in Figure 5.2.3.4. The execution time was observed to be right-skewed, indicating low latency for most routes and that outliers occur rarely. Next, the search efficiency scatter shows that most points were found below the line $y = x$, denoting that points were expanded beyond the final path. The boxplot shows the search efficiency distribution, demonstrating that no extreme outliers were detected; so, a stable searching behaviour is indicated.

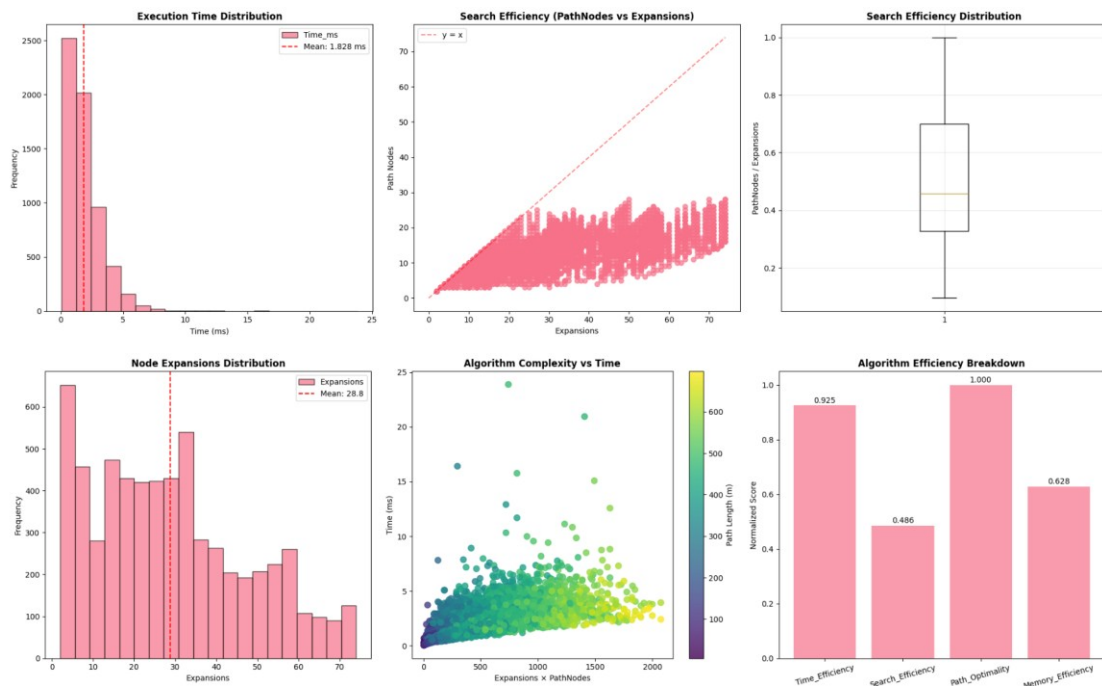


Figure 5.2.3.4: Visualization of Evaluation on Route Sets

Furthermore, a right-skewed distribution of node expansions with a long tail to about 75 expansions was observed, illustrating that most queries were handled with modest search effort; however, smaller sets were generally harder. In the complexity versus time chart, a steady increase is shown; longer paths had higher complexity and took longer time but remained within

the real-time bounds. Finally, the efficiency breakdown graph shows a strong time efficiency (92.5%) and perfect path optimality (100%), with moderate memory efficiency (62.8%) and slightly low search efficiency (48.6%).

In summary, fast and optimal paths were produced, and opportunities to reduce exploratory work and memory use were identified, providing the shortest paths using the A* algorithm to aid navigation.

5.2.4 Route Guidance Testing

Table 5.2.4.1 and Table 5.2.4.3 show the turning handling test results. Table 5.2.4.2 and Table 5.4.4 display the actual navigation instructions of the system in a screenshot.

Table 5.2.4.1: Turn Right Navigation Instruction Results

Turn Right Instructions					
Test ID/Types of Inputs	Start	Goal	Expected Output	Actual Output	Pass/Fail
ITC-001	N003	ENTRANCE_MAIN	Turn right	Turn right to Main Entrance for about 33 steps (~ 25 m).	Pass
ITC-002	N011	JUNC_MID	Turn right	Turn right to Middle Junction for about 47 steps (~ 56 m).	Pass
ITC-003	N008	N007	Turn right	Turn right to Open-Office Style Classroom for about 13 steps (~ 10 m).	Pass
ITC-004	NG-043	STAIR_L	Turn right	Turn right to Staircase Left for about 110 steps (~ 83 m).	Pass
ITC-005	NGT2	ENTRANCE_MAIN	Turn right	Turn right to Main Entrance for about 33 steps (~ 25 m).	Pass
ITC-006	N004	JUNC_MID	Turn right	Turn right to Middle Junction for about 39 steps (~ 29 m).	Pass
ITC-007	NG-036	NG-028	Turn right	Turn right to Bottom Left Junction for about 38 steps (~ 29 m).	Pass

Table 5.2.4.2: Screenshot on Turn Right Navigation Instructions

Test Case ID	Test result (Image)
ITC-008	Figure 5.2.4.1: Output for ITC-001
ITC-009	Figure 5.2.4.2: Output for ITC-002
ITC-010	Figure 5.2.4.3: Output for ITC-003
ITC-011	Figure 5.2.4.4: Output for ITC-004
ITC-012	Figure 5.2.4.5: Output for ITC-005
ITC-013	Figure 5.2.4.6: Output for ITC-006
ITC-014	Figure 5.2.4.7: Output for ITC-007

Table 5.2.4.3: Turn Left Navigation Instruction Results

Turn Left Instructions					
Test ID/Types of Inputs	Start	Goal	Expected Output	Actual Output	Pass/Fail
ITJ-001	N001	NGT3	Turn left	Turn left to Microsoft Software Engineering Laboratory for about 55 steps (~ 41 m).	Pass
ITJ-002	N007	N008	Turn left	Turn left to Microsoft Software Engineering Laboratory for about 55 steps (~ 41 m).	Pass
ITJ-003	N010	JUNC_RIGHT	Turn left	Turn left to Right Junction for about 43 steps (~ 32 m).	Pass

ITJ-004	JUNC_ LEFT	N010	Turn left	Turn left to Cisco Networking Academy Laboratory for about 47 steps (~ 35 m).	Pass
ITJ-005	N012	N001	Turn left	Turn left to Classroom N001 for about 63 steps (~ 47 m).	Pass
ITJ-006	NG-011	NG-027	Turn left	Turn left to Staffroom NG-025 for about 16 steps (~ 12m).	Pass
ITJ-007	NGT1	N001	Turn left	Turn left to Classroom N001 for about 63 steps (~ 47 m).	Pass

Table 5.2.4.4: Screenshot on Turn Left Navigation Instructions

Test Case ID	Test result (Image)
ITJ-008	Last scanned QR: Classroom N001 Destination: Disabled Toilet Suggestion: Turn left to Microsoft Software Engineering Laboratory for about 55 steps (~ 41 m). <i>Figure 5.2.4.8: Output for ITJ-001</i>
ITJ-009	Last scanned QR: Open–Office Style Classroom Destination: Microsoft Software Engineering Laboratory Suggestion: Turn left to Microsoft Software Engineering Laboratory for about 55 steps (~ 41 m). <i>Figure 5.2.4.9: Output for ITJ-002</i>
ITJ-010	Last scanned QR: Cisco Networking Academy Laboratory Destination: Right Junction Suggestion: Turn left to Right Junction for about 43 steps (~ 32 m). <i>Figure 5.2.4.10: Output for ITJ-003</i>
ITJ-011	Last scanned QR: Left Junction Destination: Cisco Networking Academy Laboratory Suggestion: Turn left to Cisco Networking Academy Laboratory for about 47 steps (~ 35 m). <i>Figure 5.2.4.11: Output for ICJ-004</i>
ITJ-012	Last scanned QR: Lab N012 Destination: Classroom N001 Suggestion: Turn left to Classroom N001 for about 63 steps (~ 47 m). <i>Figure 5.2.4.12: Output for ITJ-005</i>
ITJ-013	Last scanned QR: Staffroom NG–011 Destination: Staffroom NG–027 Suggestion: Turn left to Staffroom NG–025 for about 16 steps (~ 12 m). <i>Figure 5.2.4.13: Output for ITJ-006</i>
ITJ-014	Last scanned QR: Female Toilet Destination: Classroom N001 Suggestion: Turn left to Classroom N001 for about 63 steps (~ 47 m). <i>Figure 5.2.4.14: Output for ITJ-007</i>

5.2.5 Input Validation Testing

Table 5.2.5.1 and Table 5.2.5.2 present the results of the system handling various user inputs.

This showcases the system's ability to handle the user's unpredictable input.

Table 5.2.5.1: QR Colour Input Test Result

QR Colour Input				
Test ID/ Types of Inputs	Inputted Data	Expected Output	Actual Output	Pass/Fail
IT-001 Numeric	1	You chose colour: red	You chose colour: red	Pass
IT-002 Numeric	2	You chose colour: green	You chose colour: green	Pass
IT-003 Numeric	3	You chose colour: blue	You chose colour: blue	Pass
IT-004 Numeric	4	You chose colour: none	You chose colour: none	Pass
IT-005 Numeric	Any numerical value outside of [1,2,3,4]	Invalid choice. Please try again.	Invalid choice. Please try again.	Pass
IT-006 Text	Enter a character or a string of character	Invalid input. Please enter a number	Invalid input. Please enter a number	Pass

Table 5.2.5.2: Destination Room ID Input Test

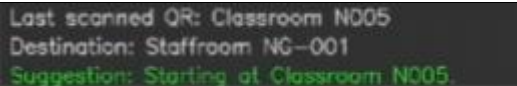
Destination room ID				
Test ID/ Types of Inputs	Inputted Data	Expected Output	Actual Output	Pass/Fail
IT-007 Text	Small cases	You chose destination: [inserted destination]	You chose destination: [inserted destination]	Pass
IT-008 Text	Capital cases	You chose destination: [inserted destination]	You chose destination: [inserted destination]	Pass
IT-009 Text	Title cases	You chose destination: [inserted destination]	You chose destination: [inserted destination]	Pass
IT-010 Text	Unlisted room ID	Destination not found. Please try again.	Destination not found. Please try again.	Pass

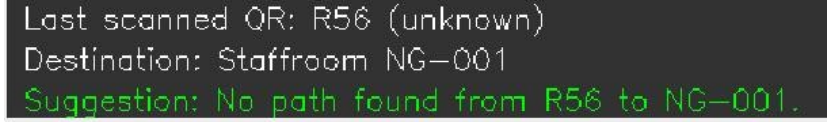
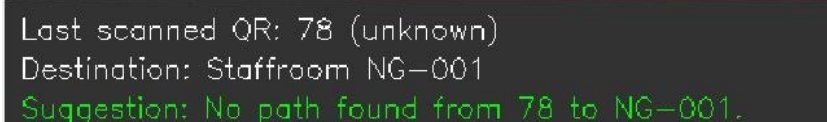
IT-011 Numeric	Any numbers	Destination not found. Please try again.	Destination not found. Please try again.	Pass
--------------------------	-------------	---	---	------

Table 5.2.5.3: Scanned QR Code Input Test

Scanned QR Code				
Test ID/ Types of Inputs	Inputted Data	Expected Output	Actual Output	Pass/Fail
IT-012 Text	Room ID in small cases	Last scanned QR: [Room ID] Suggestion: [Instruction]	Refer to Figure 5.2.4.1	Pass
IT-013 Text	Room ID in capital cases	Last scanned QR: [Room ID] Suggestion: [Instruction]	Refer to Figure 5.2.4.2	Pass
IT-014 Text	Room ID in title cases	Last scanned QR: [Room ID] Suggestion: [Instruction]	Refer to Figure 5.2.4.3	Pass
IT-015 Text	Valid room ID	Last scanned QR: [Room ID] Suggestion: [Instruction]	Refer to Figure 5.2.4.4	Pass
IT-016 Text	Invalid room ID	Last scanned QR: [decoded content] (unknown) Suggestion: No path found from [decoded content] to [destination]	Refer to 5.2.4.5, 5.2.4.6, 5.2.4.7	Pass

Table 5.2.5.4 Scanned QR Code Actual Output

Test Case ID	Test result (Image)
IT-012	 Figure 5.2.5.1: Output for IT-012
IT-013	 Figure 5.2.5.2: Output for IT-013
IT-014	

	<i>Figure 5.2.5.3: Output for IT-014</i>
IT-015	 <i>Figure 5.2.5.4: Output for IT-015</i>
IT-016	 <i>Figure 5.2.5.5: Output for IT-016 (Invalid Room ID)</i>  <i>Figure 5.2.5.6: Output for IT-016 (Numeric Value)</i>  <i>Figure 5.2.5.7: Output for IT-016 (Symbols)</i>

5.3 Project Challenges

Several challenges and limitations emerged throughout the development and testing stages, including QR code placement, user handling and interaction, dependency on a single localisation method, and hardware and performance constraints.

One of the most significant challenges faced during development was the placement of the QR codes. If positioned too high or too low, the user would have to hold the device in an awkward, uncomfortable manner. Additionally, poor QR code placement will also affect the optimal scanning distance, causing the user to stand either too close or too far from the QR code. Moreover, environmental factors such as lighting, wind, or physical obstructions could compromise the system's detection performance. For instance, the wind may fold or distort a printed QR code, rendering it unrecognisable.

Another challenge was related to user handling and interaction, whereby certain users may find it difficult to align the camera with the QR while walking. The system's ability to accurately

detect QR codes depends heavily on the angle, distance, and position with which the device is held. Although there are implementations to combat this – wider detection tolerances and real-time feedback – poor handling of the device can still undermine the system's robustness.

Variations in the user's mobility and technique also affected performance. For instance, certain users may hold the device too low or too high, tilt it excessively, or move too quickly past a QR code, resulting in a missed detection of the QR code. As the QR code serves as the sole localisation reference, and such a missed detection may cause temporary loss of localisation. However, if the user remains within the A* guided route, localisation is typically recovered at the next QR code. Therefore, effective use of the system may require a certain level of familiarity with the area or mobility skills to ensure robust performance.

5.4 Objectives Evaluation

In this chapter, the objectives of the project were evaluated to measure the effectiveness of the proposed system in addressing the needs of visually impaired users and achieving its intended goals. This evaluation was based on the testing outcomes and performance metrics that were determined during the system testing phase. This project aimed to develop an indoor navigation system for visually impaired (VI) users within the simulation area, the ground floor of the FICT building at UTAR using the coloured QR codes.

The first objective was to develop an indoor navigation system for visually impaired individuals using coloured QR codes for localisation and navigation. This objective was achieved as the prototype of the system had successfully tracked users using QR codes placed on door frames. Once the desired destination was entered, the system delivered clear and intuitive navigation instructions. During testing, the system achieved a detection success rate of over 66.67% with a decoding success rate of 83.78%. This demonstrated its functionality and reliability within the predefined area of the ground floor of the FICT building. Furthermore, the system was able

to provide up to 100% optimal path guidance; however, this proposed system was evaluated under the assumption of obstacle-free pathways.

In addition, the integration of modules and algorithms, including QR code detection, the A* algorithm pathfinding module, and the text-to-speech feature, to deliver accurate and seamless navigation was the second objective of this project. After integrating different modules together, the system was able to operate as an out-of-the-box software to provide real-time QR code detection, optimal path planning and clear navigation guidance. The integration allowed the modules to work together without significant latency and any errors. Thus, the system provided a smooth and satisfactory navigation experience to users.

5.5 Concluding Remark

Done by: Chong Jia Hui

During integrated testing, each module was validated separately to ensure that it met its objectives. The coloured-QR code detection module was shown to perform best with a coloured-on-white pattern at 256 cm² and achieved a detection rate of 66.67%. The QR code reader attained an overall decoding success rate of 83.78%, aided by grayscale and binarization. A* algorithm path planning was confirmed to be optimal on the map, with an optimality ratio of 1.000 and a median runtime of 1.505 ms⁻¹, supporting real-time shortest-path search. The route-guidance module passed all left and right turning test cases, providing clear, human-readable instructions. The user-interface input validation passed all scenarios, with numeric, text, and invalid entries. The end-to-end integration was demonstrated to operate smoothly. However, limitations observed were QR code placements, environmental factors, user handling such as device alignment, and mobility while scanning QR codes. Overall, the system was tested under an obstacle-free assumption, and it had achieved its objective of enabling indoor navigation for visually impaired users.

CHAPTER 6

Conclusion and Recommendations

6.1 Conclusion

The project “Indoor Navigation for The Visually Impaired (VI) Using QR Code” addresses the challenge of providing an efficient, low-cost indoor navigation system for VI individuals using colour-coded QR codes. The major challenge revolving this project is maintaining robust and efficient QR code detection under varying lighting conditions. The significance of this challenge is further amplified by the system’s sole reliance on QR codes for localisation, making the QR code detector a core component of the system.

Visually impaired (VI) individuals face challenges with indoor navigation, as GPS is ineffective in indoor environments and existing solutions are often costly. This project was motivated by the need for an affordable, accessible system that enhances independent and ease of movement in indoor environments. While marker-based systems exist, they often demand high computing power because the entire frames must be scanned each time to detect a QR code.

The proposed solution involves the use of a colour-coded QR code to improve detection accuracy while reducing the computational load, as colour segmentation allows the system to focus on relevant regions rather than scanning the entire frame. The system’s workflow includes QR code detection, location identification, path computation using the A* algorithm, and step-by-step navigation feedback. A novel aspect of this design is the use of colour segmentation to reduce computational load by focusing only on relevant image regions.

The integrated testing showed that the system met its objectives, with coloured-QR detection performing best using coloured-on-white patterns, a QR decoding success rate of 83.78% (enhanced by preprocessing), and the A* algorithm implemented achieved optimal real-time path planning. Route guidance and input validation modules passed all tests, while end-to-end integration ran smoothly with minimal latency. While limitations remain in QR placement,

environmental challenges, and user handling, the system successfully demonstrated indoor navigation for visually impaired users under obstacle-free conditions.

However, the system has some limitations, such as lighting shifts, glares, excessive shadow, QR code placement and accessibility, and poor camera quality. These challenges highlight areas for future improvement such as implementing image-enhancement techniques to brighten captured QR codes, integrating a vision-based model such as YOLOv5, fallback localisation such as Bluetooth beacons, and dynamic obstacle handling.

Essentially, the project successfully created an indoor navigation system simplified with the use of coloured QR code that met the initial objectives.

6.2 Future Considerations and Recommendations

Done by: Kuek Levin

Although the current system has successfully achieved its primary objectives, there are still areas for enhancement and expansion such as adopting advanced and cutting-edge technologies that could significantly improve its functionality and user experience, address the needs of VI individuals and overcome current shortcomings. This section outlines potential directions for future developments, focusing on the integration of an adaptive artificial intelligence (AI), an adaptable step counter, and multilingual feedback to address current limitations and prepare the system for broader deployment.

One of the most promising areas to explore is the integration of AI that continuously learns from user walking behaviour, preferences, and environmental interactions. The system can analyse walking speech, stride length, and route choices of the user to dynamically adapt its guidance style and timing. This not only improves the overall user experience but also enhances the robustness of the system in diverse real-world conditions. The AI builds a personalised mobility profile for each user, creating a more tailored experience.

Additionally, the current step estimation is based on general estimations, which may not accurately reflect the walking patterns of all users. An adaptable step counter can dynamically

CHAPTER 6

adjust the step length based on real-time feedback, such as terrain type, and walking speed. This will improve the accuracy of distance measurement, allowing the system to deliver more precise guidance to the user.

Currently, the text-to-speech module only supports English, restricting its use to English speakers. Expanding the system to support multilingual feedback would further enhance the system's accessibility in multilingual communities. This can be implemented with cloud-based TTS services or offline language packs, allowing users to select their preferred language during setup. This feature can also be expanded with automatic language detection based on the user's current location.

BIBLIOGRAPHY/REFERENCES

- [1] J. M. Feghali, C. Feng, A. Majumdar, and W. Y. Ochieng, “Comprehensive Review: High-Performance Positioning Systems for Navigation and Wayfinding for Visually Impaired People,” *Sensors*, vol. 24, no. 21, pp. 7020–7020, Oct. 2024.
- [2] H. A. Karimi, *Indoor Wayfinding and Navigation*. Boca Raton, FL, USA: CRC Press, 2015.
- [3] J. M. Feghali, C. Feng, A. Majumdar, and W. Y. Ochieng, “Comprehensive Review: High-Performance Positioning Systems for Navigation and Wayfinding for Visually Impaired People,” *Sensors*, vol. 20, no. 9, pp. 2641–2641, May 2020.
- [4] D. Khan, Z. Cheng, H. Uchiyama, S. Ali, M. Asshad, and K. Kiyokawa, “Recent advances in vision-based indoor navigation: A systematic literature review,” *Computers & Graphics*, vol. 104, no. 24, pp. 24–45, May 2022.
- [5] F. Zafari, A. Gkelias, and K. K. Leung, “A Survey of Indoor Localization Systems and Technologies,” *IEEE Communications Surveys Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019.
- [6] D. Wu, C. Zhang, L. Ji, R. Ran, H. Wu, and Y. Xu, “Forest Fire Recognition Based on Feature Extraction from Multi-View Images,” *Traitement Du Signal*, vol. 38, no. 3, pp. 775–783, Jun. 2021.
- [7] R. Lukac and K. N. Plataniotis, Eds., *Color Image Processing*. Boca Raton, FL, USA: CRC Press, 2018.
- [8] H. Zhang, C. Zhang, W. Yang, and C.-Y. Chen, “Localization and navigation using QR code for mobile robot in indoor environment,” *2015 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, Dec. 2015.
- [9] L. Huo, J. Zhu, P. K. Singh, and P. A. Pavlovich, “Research on QR image code recognition system based on artificial intelligence algorithm,” *Journal of Intelligent Systems*, vol. 30, no. 1, pp. 855–867, 2021.

REFERENCES

- [10] B. Li et al., “Vision-Based Mobile Indoor Assistive Navigation Aid for Blind People,” *IEEE Transactions on Mobile Computing*, vol. 18, no. 3, pp. 702–714, Mar. 2019.
- [11] E. Ko and E. Y. Kim, “A Vision-Based Wayfinding System for Visually Impaired People Using Situation Awareness and Activity-Based Instructions,” *Sensors*, vol. 17, no. 8, p. 1882, Aug. 2017.
- [12] D. Rachmawati and L. Gustin, “Analysis of Dijkstra’s algorithm and A* algorithm in shortest path problem,” *Journal of Physics Conference Series*, vol. 1566, no. 1, p. 012061, Jun. 2020.

APPENDIX A

Code Sample

A.1 App Controller Source Codes

A.1.1 Import Libraries and Define Global Variables

```
#include <opencv2/opencv.hpp>
#include <iostream>
#include <algorithm>

#include "AppController.h"
#include "../modules/TextToSpeech.h"

#include <thread>
#include <mutex>
#include <queue>
#include <condition_variable>
#include <atomic>

struct TTSTItem {
    std::string text{};
    enum class Type { Nav, Announce } type;
};

enum class TTSEngine{
    ESPEAK,
    SAPI,
    GENERIC
};

using namespace NavigationVI;

std::queue<cv::Mat> frameQueue{};
std::mutex frameMutex{};
std::condition_variable frameCV{};

std::queue<TTSTItem> ttsQueue{};
std::mutex ttsMutex{};
std::condition_variable ttsCV{};

std::atomic<bool> running{ true };

cv::Mat lastQRR0I{};

std::mutex stateMutex{};
```

```

cv::Rect lastBBox{};
std::string lastInstruction{};

std::mutex speechMutex{};
std::condition_variable speechCV{};
std::chrono::steady_clock::time_point lastSpeechEndTime{
std::chrono::steady_clock::now() };
bool speechFinished{ false };
int navCompletedCount{ 0 };

std::atomic<bool> navSpeaking{ false };
std::atomic<bool> routeReset{ false };

std::atomic<bool> newQRScanned{ false };
std::chrono::steady_clock::time_point lastQRScanTime{};

constexpr int QR_DECODE_MIN_WIDTH{ 120 };
constexpr float REF_DISTANCE_M{ 1.0f };
constexpr float REF_PIXEL_WIDTH{ 140.0f };
constexpr float TARGET_DISTANCE_M{ 0.3f };

std::chrono::steady_clock::time_point lastDistanceTTS{
std::chrono::steady_clock::now() };

TextToSpeech tts{};

```

A.1.2 Multithreading

```

void AppController::ttsWorker(TextToSpeech& tts) {
    while (running) {
        std::unique_lock<std::mutex> lock(ttsMutex);
        ttsCV.wait(lock, [] { return !ttsQueue.empty() || !running;
    });

    if (!running) break;
    TTSItem item{ std::move(ttsQueue.front()) };
    ttsQueue.pop();
    lock.unlock();

    tts.speak(item.text);
    if (item.type == TTSItem::Type::Nav) {
        std::lock_guard<std::mutex> slock(speechMutex);
        lastSpeechEndTime = std::chrono::steady_clock::now();
        navSpeaking = false;
    }
}
}

```

```

void AppController::detectionWorker(QRDetector& detector, QRReader&
reader, RouteGuidance& guider) {
    while (running) {
        cv::Mat frame{ waitForNextFrame() };
        if (frame.empty()) break;
        if (!detector.shouldAttemptDetection()) continue;

        cv::Mat hsv{};
        cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);

        cv::Mat mask{ detector.makeColourMask(hsv,
detector.getTargetColour()) };
        cv::Mat maskedFrame{};
        cv::bitwise_and(frame, frame, maskedFrame, mask);

        auto codes = detector.detectQRCodes(frame, false);
        auto nearest = detector.findNearestQRCode(codes);

        if (!nearest) {
            std::lock_guard<std::mutex> lock(stateMutex);
            lastInstruction.clear();
            lastBBox = {};
            continue;
        }

        updateGuidanceOverlay(*nearest, frame.size());

        if(!isCloseEnough(*nearest)) {
            setInstruction("Move closer to the QR");

            auto now{ std::chrono::steady_clock::now() };
            if (std::chrono::duration_cast<std::chrono::seconds>(now
- lastDistanceTTS).count() >= 2){
                {
                    std::lock_guard<std::mutex> lock(ttsMutex);
                    ttsQueue.push(TTSItem{"Please move closer to the
QR code", TTSItem::Type::Announce});
                    ttsCV.notify_one();
                }
                lastDistanceTTS = now;
            }
            continue;
        }

        if (nearest->bbox.width >= QR_DECODE_MIN_WIDTH) {
            auto roi{ extractQRROI(*nearest, frame) };
            auto content{ decodeQR(roi) };

```

```

        if (!content.empty()) {
            {
                std::lock_guard<std::mutex> lock(stateMutex);
                lastQRROI = roi.clone();
            }
            handleDecodedQR(content);
        } else {
            std::lock_guard<std::mutex> lock(stateMutex);
            lastQRROI.release();
        }
    }
}

```

A.1.3 Handle QR Code Detection

```

cv::Mat AppController::waitForNextFrame() {
    std::unique_lock<std::mutex> lock(frameMutex);
    frameCV.wait(lock, [] { return !frameQueue.empty() || !running;
});
    if (!running) return {};
    cv::Mat frame{ frameQueue.front().clone() };
    frameQueue.pop();
    return frame;
}

void AppController::queueFrame(const cv::Mat& frame) {
    std::lock_guard<std::mutex> lock(frameMutex);
    if (frameQueue.size() > 2) frameQueue.pop();
    frameQueue.push(frame.clone());
    frameCV.notify_one();
}

bool AppController::isCloseEnough(const QRCode& qr) const {
    float currentWidthPx{ static_cast<float>(qr.bbox.width) };
    float estimateDistanceM{ (REF_DISTANCE_M * REF_PIXEL_WIDTH) /
currentWidthPx };
    return estimateDistanceM <= TARGET_DISTANCE_M;
}

cv::Mat AppController::extractQRROI(const QRCode& qr, const cv::Mat&
frame) {
    if (!qr.corners.empty() && qr.corners.size() == 4) {

```

```

        double side{ 0.0f };
        for (int i = 0; i < 4; ++i) {
            double dist{ cv::norm(qr.corners[i] - qr.corners[(i + 1)
% 4]) };
            side = std::max(side, dist);
        }
        int outSide{ static_cast<int>(std::clamp(side, 16.0, 1024.0))
};

        std::vector<cv::Point2f> dst{
            {0, 0},
            {static_cast<float>(outSide - 1), 0},
            {static_cast<float>(outSide - 1),
static_cast<float>(outSide - 1)},
            {0, static_cast<float>(outSide - 1)}
        };
        cv::Mat M{ cv::getPerspectiveTransform(qr.corners, dst) };
        cv::Mat roi{};
        cv::warpPerspective(frame, roi, M, cv::Size(outSide,
outSide));
        return roi;
    } else {
        cv::Rect roiRect{ qr.bbox & cv::Rect(0, 0, frame.cols,
frame.rows) };
        return (roiRect.width > 0 && roiRect.height > 0) ?
frame(roiRect).clone() : cv::Mat{};
    }
}

```

A.1.4 Handle QR Code Decoding

```

std::string AppController::decodeQR(const cv::Mat& roi) {
    if (roi.empty()) return {};
    std::string content{ reader.reader(roi) };
    auto trim{ [](std::string& s) {
        s.erase(s.begin(), std::find_if(s.begin(), s.end(),
            [](unsigned char ch){ return !std::isspace(ch); }));
        s.erase(std::find_if(s.rbegin(), s.rend(),
            [](unsigned char ch){ return !std::isspace(ch);
        }).base(), s.end());
    } };
    trim(content);
    toUpperInPlace(content);
    return content;
}

void AppController::handleDecodedQR(const std::string& content) {

```

```

std::string prevQR;
{
    std::lock_guard<std::mutex> lock(stateMutex);
    prevQR = lastQRData;
    lastQRData = content;
    if (auto resolved = mapSystem.resolveRoomId(content))
        lastRoomName =
mapSystem.getRooms().at(resolved.value()).m_name;
    else
        lastRoomName = content + " (unknown)";
}
if (content != prevQR) {
    if (currentInstructions.empty()) {
        handleNewQR(content);
    } else {
        for (size_t i = 0; i < currentInstructions.size(); ++i) {
            if (currentInstructions[i].text.find(content) !=
std::string::npos) {
                if (i + 1 < currentInstructions.size()) {
                    currentStepIndex = i+1;
                    currentSuggestion =
currentInstructions[currentStepIndex].text;
                } else {
                    currentStepIndex = i;
                    currentSuggestion =
currentInstructions[i].text;

                    std::lock_guard<std::mutex> qlock(ttsMutex);
                    ttsQueue.push(TTSItem{currentSuggestion,
TTSItem::Type::Nav});
                    ttsCV.notify_one();
                }
                break;
            }
        }
    }

    {
        std::lock_guard<std::mutex> lock(stateMutex);
        newQRScanned = true;
        lastQRScanTime = std::chrono::steady_clock::now();
        lastQRData = content;
    }

    std::lock_guard<std::mutex> lock(ttsMutex);
    ttsQueue.push(TTSItem{"QR detected: " + content,
TTSItem::Type::Announce});
    ttsCV.notify_one();
}

```

```

        if (!lastInstruction.empty()) {
            ttsQueue.push(TTSItem{lastInstruction,
TTSItem::Type::Nav});
            ttsCV.notify_one();
        }
        else if (!currentInstructions.empty() && currentStepIndex ==
currentInstructions.size() - 1) {
            std::cout << currentSuggestion;
            ttsQueue.push(TTSItem{currentSuggestion,
TTSItem::Type::Nav});
            ttsCV.notify_one();
        }
    }
}

```

A.1.5 Handle Route Guidance

```

void AppController::maybeAdvanceStep() {
    std::string nextText{};
    bool canAdvance{false};
    {
        std::lock_guard<std::mutex> lock(stateMutex);
        if (newQRScanned && !currentInstructions.empty() &&
            currentStepIndex + 1 < currentInstructions.size() - 1) {

            auto now = std::chrono::steady_clock::now();
            if (!navSpeaking && (now - lastQRScanTime >=
std::chrono::seconds(3))) {
                canAdvance = true;
                nextText = currentInstructions[currentStepIndex +
1].text;
                newQRScanned = false;
            }
        }
        if (canAdvance) {
            navSpeaking = true;
            {
                std::lock_guard<std::mutex> qlock(ttsMutex);
                ttsQueue.push(TTSItem{nextText, TTSItem::Type::Nav});
                ttsCV.notify_one();
            }
            {
                std::lock_guard<std::mutex> lock(stateMutex);
                ++currentStepIndex;
            }
        }
    }
}

```

```

        currentSuggestion = nextText;
        m_firstStepAfterQR = false;
    }
}

void AppController::setInstruction(const std::string& text) {
    std::lock_guard<std::mutex> lock(stateMutex);
    lastInstruction = text;
}

```

A.1.6 Handle User Interface

```

void AppController::updateGuidanceOverlay(const QRCode& qr, const
cv::Size& frameSize) {
    auto cmd{ detector.getNavigationToQR(qr, frameSize) };
    std::lock_guard<std::mutex> lock(stateMutex);
    lastInstruction = cmd.instruction;
    lastBBox = qr.bbox;
}

void AppController::drawOverlay(cv::Mat& frame) {
    std::lock_guard<std::mutex> lock(stateMutex);
    if (!lastInstruction.empty())
        cv::putText(frame, lastInstruction, {10, 30},
            cv::FONT_HERSHEY_SIMPLEX, 0.7, {0, 255, 0}, 2);
    if (lastBBox.area() > 0)
        cv::rectangle(frame, lastBBox, {0, 255, 0}, 2);
}

void AppController::showComposite(const cv::Mat& frame) {
    cv::Mat hsv{}, mask{};
    cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
    mask = detector.makeColourMask(hsv, detector.getTargetColour());
    cv::Mat qrPreview{};
    {
        std::lock_guard<std::mutex> lock(stateMutex);
        qrPreview = lastQRROI.empty() ? cv::Mat(frame.size(),
frame.type(), cv::Scalar(0,0,0)) : lastQRROI.clone();
        qrPreview = lastQRROI.clone();
    }

    cv::Mat composite{ ui.makeComposite(frame, mask, qrPreview) };
    cv::Mat finalDisplay{ ui.addTextPanel(composite, lastRoomName,
destinationName, currentSuggestion) };
    ui.showWindow(finalDisplay);
}

```


A.1.7 App Controller Header Files

```

#pragma once
#include <string>
#include <vector>
#include <chrono>
#include "../modules/QRDetector.h"
#include "../modules/QRReader.h"
#include "../modules/CoordinateMapSystem.h"
#include "../modules/RouteGuidance.h"
#include "../modules/TextToSpeech.h"
#include "UIManager.h"

namespace NavigationVI{
    class AppController{
    public:
        AppController();

        void ttsWorker(TextToSpeech& tts);
        void detectionWorker(QRDetector& detector, QRReader& reader,
RouteGuidance& guider);

        cv::Mat waitForNextFrame();
        void queueFrame(const cv::Mat& frame);
        bool isCloseEnough(const QRCode& qr) const;
        cv::Mat extractQRROI(const QRCode& qr, const cv::Mat& frame);

        std::string decodeQR(const cv::Mat& roi);
        void handleDecodedQR(const std::string& content);

        void maybeAdvanceStep();
        void setInstruction(const std::string& text);

        void updateGuidanceOverlay(const QRCode& qr, const cv::Size&
frameSize);
        void drawOverlay(cv::Mat& frame);
        void showComposite(const cv::Mat& frame);

        bool checkForExitKey();
        void run();
    public:
        bool m_firstStepAfterQR{};
        cv::Mat lastQRROI{};
    private:
        void handleNewQR(const std::string& content);
    private:
        QRDetector detector;
        QRReader reader;

```

```

CoordinateMapSystem mapSystem;
RouteGuidance guider;
UIManager ui;

std::string lastQRData{};
std::string lastRoomName{};
std::string destinationId{};
std::string destinationName{};
std::string currentSuggestion{};
std::vector<Instruction> currentInstructions{};
size_t currentStepIndex{};
std::chrono::steady_clock::time_point lastStepTime{};
const std::chrono::seconds stepInterval{ 8 };

double unitScale{ 1.0 };
double stepLengthM{ 0.75 };
};
}

```

A.2 QR Code Detection Source Codes

A.2.1 QR Code Detection Source Code (QRDetector.h)

```

#pragma once

#include <string>
#include <vector>
#include <optional>
#include <map>
#include <chrono>

#include <opencv2/opencv.hpp>

#include "../utils/QRCode.h"

namespace NavigationVI{

    class QRDetector{
    public:
        QRDetector();

        bool initialise(const int cameraIndex = 0);
        void release();
    };
}

```

```

        void setTargetColour(const QRColour colour);
        QRColour getTargetColour() const;

        void setDetectionThrottle(int framesInterval, int
minGapMs);
        bool shouldAttemptDetection();

        std::vector<QRCode> detectQRCodes(const cv::Mat& frame,
bool tryDecode = false);
        std::optional<QRCode> findNearestQRCode(const
std::vector<QRCode>& codes) const;
        NavigationCommand getNavigationToQR(const QRCode& qr,
const cv::Size& frameSize) const;

        void setMinArea(int area);
        void setAspectRatioTolerance(float low, float high);
        void setBoundingBoxPadding(int px);
        void setDistanceReference(float pxAt1m, float meters);

        void setColourVerificationEnabled(bool enabled);
        bool getColourVerificationEnabled() const;
        cv::Mat makeColourMask(const cv::Mat& hsv, QRColour
colour) const;

        // optional: Can delete later
        public:
            bool isOpened() const { return m_camera.isOpened(); }
            bool readFrame(cv::Mat& out) { return m_camera.read(out); }
    }

    private:
        cv::VideoCapture m_camera{};
        QRColour m_targetColour{ QRColour::NONE };

        int m_minAreaPx{ 1000 };
        float m_aspectRatioLow{ 0.6f };
        float m_aspectRatioHigh{ 1.8f };
        int m_bbboxPadding{ 25 };
        float m_referencePx{ 120.0f };
        float m_referenceMeters{ 1.0f };

        bool m_colourVerifyEnabled{ true };

        int m_detectionIntervalFrames{ 5 };
        int m_minDetectionGapMs{ 500 };
        int m_frameSkipCounter{ 0 };

```

```

        std::chrono::steady_clock::time_point
m_lastDetectionTime{};

        std::map<QRColour, std::vector<HSVRange>>
m_colourRanges{};

        std::vector<cv::Rect> findCandidateROIs(const cv::Mat&
mask) const;

        DetectResult robustDetectInROI(const cv::Mat& frame,
const cv::Rect& roi, bool tryDecode) const;

        static cv::Rect padRect(const cv::Rect& r, int pad, const
cv::Size& maxSize);
        static bool isAspectOk(const cv::Rect& r, float low,
float high);
        static float estimateDistancePxToMeters(float bboxPx,
float referencePx, float referenceMeters);
        static float averageSide(const cv::Rect& r);
        bool verifyColourInROI(const cv::Mat& frame, const
cv::Rect& roi, QRColour colour) const;
    };
}

```

A.2.1 QR Code Detection Source Code (QRDetector.cpp)

```

#include "QRDetector.h"

#include <algorithm>
#include <cmath>

namespace NavigationVI {
    static inline bool minRoiOk(const cv::Rect& r, int minSide = 16)
    {
        return r.width >= minSide && r.height >= minSide;
    }

    static inline bool polygonIsSane(const std::vector<cv::Point2f>&
pts) {
        if (pts.size() != 4) return false;

        double area = std::fabs(cv::contourArea(pts));
        if (area < 25.0) return false;

        std::vector<cv::Point> polyInt; polyInt.reserve(4);
    }
}

```

```

        for (auto& p : pts)
polyInt.emplace_back(cv::Point(cv::saturate_cast<int>(p.x),
        cv::saturate_cast<int>(p.y)));
        if (!cv::isContourConvex(polyInt)) return false;

        double d01 = cv::norm(pts[0] - pts[1]);
        double d12 = cv::norm(pts[1] - pts[2]);
        double d23 = cv::norm(pts[2] - pts[3]);
        double d30 = cv::norm(pts[3] - pts[0]);
        double longest = std::max(std::max(d01, d12), std::max(d23,
d30));
        double shortest = std::min(std::min(d01, d12), std::min(d23,
d30));
        if (shortest < 1.0 || longest / shortest > 4.0) return false;

        return true;
    }

    static inline cv::Rect bboxFromCorners(const
std::vector<cv::Point2f>& corners,
        int pad,
        const cv::Size& frameSize) {
        cv::Rect r = cv::boundingRect(corners);
        r.x = std::max(0, r.x - pad);
        r.y = std::max(0, r.y - pad);
        r.width += 2 * pad;
        r.height += 2 * pad;
        r &= cv::Rect(0, 0, frameSize.width, frameSize.height);
        return r;
    }

    QRDetector::QRDetector()
        : m_targetColour(QRColour::NONE),
        m_minAreaPx(1000),
        m_aspectRatioLow(0.6f),
        m_aspectRatioHigh(1.6f),
        m_bbboxPadding(25),
        m_referencePx(120.0f),
        m_referenceMeters(1.0f),
        m_colourVerifyEnabled(true),
        m_detectionIntervalFrames(5),
        m_minDetectionGapMs(500),
        m_frameSkipCounter(0),
        m_lastDetectionTime(std::chrono::steady_clock::now()) {

        m_colourRanges[QRColour::BLUE] = {
            {
                cv::Scalar(95, 60, 50), cv::Scalar(140, 255, 255)
            }
        }
    }

```

```

    }
};

m_colourRanges[QRColour::GREEN] = {
    {
        cv::Scalar(35, 60, 50), cv::Scalar(85, 255, 255)
    }
};

m_colourRanges[QRColour::RED] = {
    {
        cv::Scalar(0, 60, 50), cv::Scalar(10, 255, 255)
    },
    {
        cv::Scalar(170, 60, 50), cv::Scalar(179, 255, 255)
    }
};
}

// Destructor is not needed

bool QRDetector::initialise(int cameraIndex) {
    m_camera.open(cameraIndex);
    return m_camera.isOpened();
}

void QRDetector::release() {
    if (m_camera.isOpened()) m_camera.release();
}

void QRDetector::setDetectionThrottle(int framesInterval, int
minGapMs) {
    m_detectionIntervalFrames = std::max(1, framesInterval);
    m_minDetectionGapMs = std::max(0, minGapMs);
}

bool QRDetector::shouldAttemptDetection() {
    ++m_frameSkipCounter;
    auto now{ std::chrono::steady_clock::now() };
    auto ms{
std::chrono::duration_cast<std::chrono::milliseconds>(now -
m_lastDetectionTime).count() };

    bool intervalPassed{ m_frameSkipCounter >=
m_detectionIntervalFrames };
    bool cooldownPassed{ ms >= m_minDetectionGapMs };

    if (intervalPassed && cooldownPassed) {

```

```

        m_frameSkipCounter = 0;
        m_lastDetectionTime = now;
        return true;
    }
    return false;
}

cv::Mat QRDetector::makeColourMask(const cv::Mat& hsv, QRColour
colour) const {
    cv::Mat mask{ cv::Mat::zeros(hsv.size(), CV_8UC1) };
    if (colour == QRColour::NONE) {
        mask.setTo(255);
        return mask;
    }

    auto it{ m_colourRanges.find(colour) };
    if (it == m_colourRanges.end()) return mask;

    for (const auto& rng : it->second) {
        cv::Mat part{};
        cv::inRange(hsv, rng.lower, rng.upper, part);
        cv::bitwise_or(mask, part, mask);
    }

    cv::Mat kernel{ cv::getStructuringElement(cv::MORPH_RECT,
{3,3}) };
    cv::morphologyEx(mask, mask, cv::MORPH_OPEN, kernel);
    cv::morphologyEx(mask, mask, cv::MORPH_CLOSE, kernel);

    return mask;
}

std::vector<cv::Rect> QRDetector::findCandidateROIs(const
cv::Mat& mask) const {
    const double scale{ 0.5 };
    cv::Mat smallMask;
    cv::resize(mask, smallMask, cv::Size(), scale, scale,
cv::INTER_NEAREST);

    std::vector<std::vector<cv::Point>> contours;
    cv::findContours(smallMask, contours, cv::RETR_EXTERNAL,
cv::CHAIN_APPROX_SIMPLE);

    std::vector<cv::Rect> rois;
    rois.reserve(contours.size());

    for (const auto& c : contours) {
        double area = cv::contourArea(c);

```

```

        if (area < static_cast<double>(m_minAreaPx) * scale *
scale) continue;

        cv::Rect bb = cv::boundingRect(c);

        // Scale ROI back to full resolution
        cv::Rect fullBB(
            static_cast<int>(bb.x / scale),
            static_cast<int>(bb.y / scale),
            static_cast<int>(bb.width / scale),
            static_cast<int>(bb.height / scale)
        );

        if (!isAspectOk(fullBB, m_aspectRatioLow,
m_aspectRatioHigh)) continue;

        rois.push_back(fullBB);
    }

    std::sort(rois.begin(), rois.end(),
        [](const cv::Rect& a, const cv::Rect& b) {
            return a.area() > b.area();
        });

    return rois;
}

DetectResult QRDetector::robustDetectInROI(
    const cv::Mat& frame,
    const cv::Rect& roi,
    bool tryDecode
) const {

    DetectResult out{};

    cv::Rect padded{ padRect(roi, m_bbboxPadding, frame.size())
};

    padded &= cv::Rect{ 0, 0, frame.cols, frame.rows };
    if (padded.width < 2 || padded.height < 2) return out;

    cv::Mat roiBGR{ frame(padded).clone() };
    if (roiBGR.empty()) return out;

    cv::Mat gray{};
    cv::cvtColor(roiBGR, gray, cv::COLOR_BGR2GRAY);

    cv::QRCodeDetector qrd{};

```



```

        auto attempt = [&](const cv::Mat& img, bool decode,
DetectResult& result) -> bool {
            if (img.empty() || img.cols < 2 || img.rows < 2) return
false;

            std::vector<cv::Point> pts{};
            std::string data{};

            try {
                if (decode) data = qrd.detectAndDecode(img, pts);
                else if (!qrd.detect(img, pts)) return false;
            }
            catch (const cv::Exception& e) {
                std::cerr << "robustDetectInROI detect error: " <<
e.what() << std::endl;
                return false;
            }

            if (pts.size() == 4) {
                std::vector<cv::Point2f> corners{};
                corners.reserve(4);
                // result.corners.clear();
                for (auto& p : pts) {
                    corners.emplace_back(
                        static_cast<float>(p.x + padded.x),
                        static_cast<float>(p.y + padded.y)
                    );
                }
                if (!polygonIsSane(corners)) return false;

                cv::Rect paddedBox{ bboxFromCorners(corners, 20,
frame.size()) };
                if (!minRoiOk(paddedBox, 16)) return false;

                result.corners = std::move(corners);
                result.bbox = paddedBox;

                result.content = decode ? data : std::string{};
                result.found = decode ? (!data.empty()) : true;
                return result.found;
            }
            return false;
        };

        if (attempt(gray, tryDecode, out)) return out;

        cv::Mat invGray{}; cv::bitwise_not(gray, invGray);
        if (attempt(invGray, tryDecode, out)) return out;

```

```

        {
            std::vector<cv::Mat> ch{};
            cv::split(roiBGR, ch);
            for (auto& c : ch) {
                cv::Mat inv{};
                cv::bitwise_not(c, inv);
                if (attempt(inv, tryDecode, out)) return out;
            }
        }

        cv::Mat otsu{};
        cv::threshold(gray, otsu, 0, 255, cv::THRESH_BINARY |
cv::THRESH_OTSU);

        cv::Mat invOtsu{}; cv::bitwise_not(otsu, invOtsu);
        if (attempt(invOtsu, tryDecode, out)) return out;

        cv::Mat adap{};
        cv::adaptiveThreshold(gray, adap, 255,
cv::ADAPTIVE_THRESH_GAUSSIAN_C,
            cv::THRESH_BINARY, 11, 2);
        cv::Mat invAdap{}; cv::bitwise_not(adap, invAdap);
        if (attempt(invAdap, tryDecode, out)) return out;

        return out;
    }

    cv::Rect QRDetector::padRect(const cv::Rect& r, int pad, const
cv::Size& maxSize) {
        int x{ std::max(0, r.x - pad) };
        int y{ std::max(0, r.y - pad) };
        int x2{ std::min(maxSize.width, r.x + r.width + pad) };
        int y2{ std::min(maxSize.height, r.y + r.height + pad) };
        return cv::Rect(x, y, x2 - x, y2 - y);
    }

    bool QRDetector::isAspectOk(const cv::Rect& r, float low, float
high) {
        float ar{ static_cast<float>(r.width) / std::max(1, r.height)
};
        return (ar >= low && ar <= high);
    }

    float QRDetector::averageSide(const cv::Rect& r) {
        return 0.5f * (static_cast<float>(r.width) +
static_cast<float>(r.height));
    }

```

```

    bool QRDetector::verifyColourInROI(const cv::Mat& frame, const
cv::Rect& roi, QRColour colour) const {
    if (colour == QRColour::NONE) return true;
    cv::Mat bgr{ frame(roi) };
    cv::Mat hsv{}; cv::cvtColor(bgr, hsv, cv::COLOR_BGR2HSV);

    auto it{ m_colourRanges.find(colour) };
    if (it == m_colourRanges.end()) return false;

    cv::Mat accum{ cv::Mat::zeros(hsv.size(), CV_8UC1) };
    for (const auto& rng : it->second) {
        cv::Mat part{};
        cv::inRange(hsv, rng.lower, rng.upper, part);
        cv::bitwise_or(accum, part, accum);
    }

    double ratio{ static_cast<double>(cv::countNonZero(accum)) /
(accum.rows * accum.cols + 1e-6) };
    return ratio > 0.25;
}

float QRDetector::estimateDistancePxToMeters(float bboxPx, float
referencePx, float referenceMeters) {
    if (bboxPx <= 1.0f) return
std::numeric_limits<float>::infinity();
    return (referencePx / bboxPx) * referenceMeters;
}

std::vector<QRCode> QRDetector::detectQRCodes(const cv::Mat&
frame, bool tryDecode) {
    std::vector<QRCode> out{};
    if (frame.empty()) return out;

    cv::Mat hsv{}; cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
    cv::Mat mask{ makeColourMask(hsv, m_targetColour) };
    auto rois{ findCandidateROIs(mask) };

    for (const auto& roi : rois) {
        auto det{ robustDetectInROI(frame, roi, tryDecode) };
        if (!det.found) continue;

        cv::Rect box{};
        if (det.corners.size() == 4)
            box = bboxFromCorners(det.corners, m_bbboxPadding,
frame.size());
        else {
            box = det.bbox.area() > 0 ? det.bbox : roi;

```

```

        box &= cv::Rect(0, 0, frame.cols, frame.rows);
    }

    if (!minRoiOk(box, 16)) continue;
    if (m_colourVerifyEnabled && !verifyColourInROI(frame,
box, m_targetColour)) continue;

    QRCode qr{};
    qr.position = cv::Point2f{
        box.x + box.width * 0.5f,
        box.y + box.height * 0.5f
    };
    qr.content = tryDecode ? det.content : std::string{};
    qr.colour = m_targetColour;
    float px{ averageSide(box) };
    qr.distance = estimateDistancePxToMeters(px,
m_referencePx, m_referenceMeters);
    qr.bbox = box;
    qr.corners = det.corners;

    out.push_back(qr);
}

if (out.empty() && m_targetColour == QRColour::NONE) {
    cv::Rect full{ 0, 0, frame.cols, frame.rows };
    auto det{ robustDetectInROI(frame, full, tryDecode) };
    if (det.found) {
        cv::Rect box{};
        if (det.corners.size() == 4) box =
bboxFromCorners(det.corners, m_bbboxPadding, frame.size());
        else {
            box = det.bbox.area() ? det.bbox : full;
            box &= cv::Rect(0, 0, frame.cols, frame.rows);
        }
        if (minRoiOk(box, 16)) {
            QRCode qr{};
            qr.position = cv::Point2f{
                box.x + box.width * 0.5f,
                box.y + box.height * 0.5f
            };
            qr.colour = QRColour::NONE;
            float px{ averageSide(box) };
            qr.distance = estimateDistancePxToMeters(px,
m_referencePx, m_referenceMeters);
            qr.bbox = box;
            qr.corners = det.corners;
            qr.content = tryDecode ? det.content :
std::string{};

```

```

        out.push_back(qr);
    }
}

return out;
}

std::optional<QRCode> QRDetector::findNearestQRCode(const
std::vector<QRCode>& codes) const {
    if (codes.empty()) return std::nullopt;
    return *std::min_element(codes.begin(), codes.end(),
        [](const QRCode& a, const QRCode& b) {return a.distance <
b.distance; });
}

NavigationCommand QRDetector::getNavigationToQR(const QRCode& qr,
const cv::Size& frameSize) const {
    cv::Point2f center(frameSize.width * 0.5f, frameSize.height *
0.5f);
    cv::Point2f delta{ qr.position - center };

    const float hfovDeg{ 60.0f };
    float normX{ (delta.x) / std::max(1.0f, frameSize.width *
0.5f) };
    float angleDeg{ normX * (hfovDeg * 0.5f) };

    NavigationCommand cmd{};
    cmd.distance = qr.distance;
    cmd.angle = std::fabs(angleDeg);

    const float straightThresh{ 5.0f };
    const float ninetyThresh{ 45.0f };

    if (std::fabs(angleDeg) <= straightThresh) {
        cmd.direction = Direction::FORWARD;
        cmd.instruction = "Move forward";
    }
    else if (angleDeg > 0) {
        if (cmd.angle >= ninetyThresh) {
            cmd.direction = Direction::TURN_RIGHT_90;
            cmd.instruction = "Turn right about 90 degrees";
        }
        else {
            cmd.direction = Direction::TURN_RIGHT_45;
            cmd.instruction = "Turn right " +
std::to_string(static_cast<int>(cmd.angle)) + " degrees";
        }
    }
}

```

```

        else {
            if (cmd.angle >= ninetyThresh) {
                cmd.direction = Direction::TURN_LEFT_90;
                cmd.instruction = "Turn left about 90 degrees";
            }
            else {
                cmd.direction = Direction::TURN_LEFT_45;
                cmd.instruction = "Turn left " +
std::to_string(static_cast<int>(cmd.angle)) + " degrees";
            }
        }
        return cmd;
    }

    void QRDetector::setTargetColour(QRColour colour) {
m_targetColour = colour; }
    QRColour QRDetector::getTargetColour() const { return
m_targetColour; }
    void QRDetector::setMinArea(int area) { m_minAreaPx = area; }
    void QRDetector::setAspectRatioTolerance(float low, float high) {
m_aspectRatioLow = low; m_aspectRatioHigh = high; }
    void QRDetector::setBoundingBoxPadding(int px) { m_bbboxPadding =
px; }
    void QRDetector::setDistanceReference(float pxAt1m, float meters)
{ m_referencePx = pxAt1m; m_referenceMeters = meters; }
    void QRDetector::setColourVerificationEnabled(bool enabled) {
m_colourVerifyEnabled = enabled; }
    bool QRDetector::getColourVerificationEnabled() const { return
m_colourVerifyEnabled; }
}

```

A.3 QR Code Reader Source Codes

A.3.1 QR Code Reader Source Code File (QRReader.cpp)

```

#include "QRReader.h"

#include <string>
#include <iostream>
#include <algorithm>
#include <vector>

namespace NavigationVI {

```

```

static inline bool tooSmall(const cv::Mat& img, int minSide = 8)
{
    return img.empty() || img.cols < minSide || img.rows <
minSide;
}

cv::Mat QRReader::toGray(const cv::Mat& image) const {
    if (image.empty()) return {};
    if (image.channels() == 1) return image;
    cv::Mat gray{};
    if (image.channels() == 4) cv::cvtColor(image, gray,
cv::COLOR_RGBA2GRAY);
    else cv::cvtColor(image, gray, cv::COLOR_BGR2GRAY);

    return gray;
}

std::string QRReader::printResult(const
std::vector<zbar::Symbol>& zbarResults) const {
    std::string data{};
    for (const auto& symbol : zbarResults) {
        std::string msg = "QR detected: " + symbol.get_data();
        std::cout << msg << std::endl;
        if (onMessage) onMessage(msg);
        data = symbol.get_data();
    }
    return data;
}

std::vector<zbar::Symbol> QRReader::decodeWithZbar(const cv::Mat&
image, zbar::ImageScanner& scanner) const {
    std::vector<zbar::Symbol> results{};
    if (tooSmall(image, 8)) return results;
    cv::Mat gray{};
    if (image.channels() == 1) gray = image;
    else cv::cvtColor(image, gray, cv::COLOR_BGR2GRAY);
    if (tooSmall(gray, 8)) return results;

    zbar::Image zbarImage(
        gray.cols,
        gray.rows,
        "Y800",
        static_cast<void*>(const_cast<uchar*>(gray.data)),
        gray.cols * gray.rows
    );

    int n{ scanner.scan(zbarImage) };
    if (n > 0) {
        for (zbar::Image::SymbolIterator
symbol{zbarImage.symbol_begin()};
            symbol != zbarImage.symbol_end(); ++symbol)
        {
            results.push_back(*symbol);
        }
    }
}

```

```

        return results;
    }

    std::string QRReader::decodeQR(const cv::Mat& image,
    zbar::ImageScanner& scanner, const std::string& stageName) const {
        if (tooSmall(image, 8)) return "";
        std::vector<zbar::Symbol> results{ decodeWithZbar(image,
    scanner) };
        if (!results.empty()) {
            std::cout << "\nZbar detected " << results.size()
                << " Qr code(s) in " << stageName << std::endl;
            return printResult(results);
        }
        return "";
    }

    cv::Mat QRReader::thresholdImage(const cv::Mat& image) const {
        if (tooSmall(image, 8)) return image;
        cv::Mat threshold{};
        cv::threshold(image, threshold, 0, 255, cv::THRESH_BINARY |
    cv::THRESH_OTSU);
        return threshold;
    }

    std::string QRReader::reader(const cv::Mat& imageBGR) {
        if (tooSmall(imageBGR, 12)) {
            std::cout << "QRReader: empty or too small image,
    skipping." << std::endl;
            return "";
        }

        zbar::ImageScanner scanner{};
        scanner.set_config(zbar::ZBAR_NONE, zbar::ZBAR_CFG_ENABLE,
    1);

        {
            std::string result{ decodeQR(imageBGR, scanner, "original
    image") };
            if (!result.empty()) return result;
        }

        cv::Mat gray{ toGray(imageBGR) };
        if (tooSmall(gray, 12)) return "";

        {
            std::string result{ decodeQR(gray, scanner, "gray image")
    };
            if (!result.empty()) return result;
        }

        cv::Mat thresholded{ thresholdImage(gray) };
        if (!tooSmall(thresholded, 8)) {
            std::string result = decodeQR(thresholded, scanner,
    "thresholded image");
            if (!result.empty()) return result;
        }
    }

```



```

    }

    std::cout << "\nZBar: No QR code detected" << std::endl;
    // if (onMessage) onMessage("No QR code detected");
    return "";
}
}

```

A.3.2 QR Code Reader Header Source Code (QRReader.h)

```

#pragma once

#include <string>
#include <vector>
#include <functional>

#include <opencv2/opencv.hpp>
#include <zbar.h>

namespace NavigationVI {
    class QRReader {
    public:
        std::string reader(const cv::Mat& imageBGR);
    public:
        std::function<void(const std::string&)> onMessage{};
    private:
        cv::Mat toGray(const cv::Mat& image) const;
        std::string printResult(const std::vector<zbar::Symbol>&
zbarResults) const;
        std::vector<zbar::Symbol> decodeWithZbar(const cv::Mat&
image, zbar::ImageScanner& scanner) const;
        std::string decodeQR(const cv::Mat& image,
zbar::ImageScanner& scanner,
const std::string& stageName) const;
        cv::Mat thresholdImage(const cv::Mat& image) const;
    };
}

```

A.4 Map Creation and Path Planning Source Codes

A.4.1 rooms.txt

```

# ----- Classrooms -----
N007|Open-Office Style Classroom|CLASSROOM|20|80|20|15
N006|Classroom N006|CLASSROOM|45|80|20|15
N005|Classroom N005|CLASSROOM|70|80|20|15
N004|Classroom N004|CLASSROOM|95|80|20|15
N003|Classroom N003|CLASSROOM|120|80|20|15

```

APPENDIX A

```

N002|Classroom N002|CLASSROOM|145|80|20|15
N001|Classroom N001|CLASSROOM|170|80|20|15

# ----- Laboratories -----
N008|Microsoft Software Engineering Laboratory|LABORATORY|20|40|20|15
N009|Silverlab|LABORATORY|95|40|20|15
N010|Cisco Networking Academy Laboratory|LABORATORY|120|40|20|15
N011|IPSR Lab|LABORATORY|145|40|20|15
N012|Lab N012|LABORATORY|170|40|20|15

# ----- Toilets -----
NGT1|Female Toilet|TOILET|195|40|8|8
NGT2|Ladies Toilet|TOILET|205|40|8|8
NGT3|Disabled Toilet|TOILET|45|40|8|8
NGT4|Male Toilet|TOILET|70|40|8|8
NGT5|Female Toilet|TOILET|80|40|8|8

# ----- Entrances -----
ENTRANCE_MAIN|Main Entrance|ENTRANCE|95|90|10|6

# ----- Junctions -----
JUNC_LEFT|Left Junction|CORRIDOR|10|80|4|4
JUNC_MID|Middle Junction|CORRIDOR|95|65|4|4
JUNC_RIGHT|Right Junction|CORRIDOR|215|65|4|4
JUNC_TOP_LEFT|Top Left Junction|CORRIDOR|10|120|4|4
JUNC_BOTTOM_LEFT|Bottom Left Junction|CORRIDOR|215|40|4|4

# ----- Staffrooms NG-001..NG-014 (reverse order) -----
NG-001|Staffroom NG-001|OFFICE|5|122|14|14
NG-002|Staffroom NG-002|OFFICE|30|120|14|14
NG-003|Staffroom NG-003|OFFICE|45|120|14|14
NG-004|Staffroom NG-004|OFFICE|60|120|14|14
NG-005|Staffroom NG-005|OFFICE|75|120|14|14
NG-006|Staffroom NG-006|OFFICE|90|120|14|14
NG-007|Staffroom NG-007|OFFICE|105|120|14|14
NG-008|Staffroom NG-008|OFFICE|120|120|14|14
NG-009|Staffroom NG-009|OFFICE|135|120|14|14
NG-010|Staffroom NG-010|OFFICE|150|120|14|14
NG-011|Staffroom NG-011|OFFICE|165|120|14|14
NG-012|Staffroom NG-012|OFFICE|180|120|14|14
NG-013|Staffroom NG-013|OFFICE|195|120|14|14
NG-014|Staffroom NG-014|OFFICE|210|120|14|14

# ----- Staffrooms NG-015..NG-023 (positions reversed: 15 on the
right, 23 on the left) -----
NG-015|Staffroom NG-015|OFFICE|135|90|14|14
NG-016|Staffroom NG-016|OFFICE|120|90|14|14

```

APPENDIX A

```

NG-017|Staffroom NG-017|OFFICE|105|90|14|14
NG-018|Staffroom NG-018|OFFICE|90|90|14|14
NG-019|Staffroom NG-019|OFFICE|75|90|14|14
NG-020|Staffroom NG-020|OFFICE|60|90|14|14
NG-021|Staffroom NG-021|OFFICE|45|90|14|14
NG-022|Staffroom NG-022|OFFICE|30|90|14|14
NG-023|Staffroom NG-023|OFFICE|15|90|14|14

# ----- NG-024..NG-030 precise L-shape -----
NG-024|Staffroom NG-024|OFFICE|-13|90|14|14
NG-025|Staffroom NG-025|OFFICE|-13|78|14|14
NG-026|Staffroom NG-026|OFFICE|-13|66|14|14
NG-027|Staffroom NG-027|OFFICE|-13|54|14|14
NG-028|Staffroom NG-028|OFFICE|-13|42|14|14
NG-029|Staffroom NG-029|OFFICE|9|54|14|14
NG-030|Staffroom NG-030|OFFICE|9|42|14|14

# ----- NG-031..NG-043 : bottom row (keep standard left-right) -----
-----
NG-031|Staffroom NG-031|OFFICE|15|60|14|14
NG-032|Staffroom NG-032|OFFICE|30|60|14|14
NG-033|Staffroom NG-033|OFFICE|45|60|14|14
NG-034|Staffroom NG-034|OFFICE|60|60|14|14
NG-035|Staffroom NG-035|OFFICE|75|60|14|14
NG-036|Staffroom NG-036|OFFICE|90|60|14|14
NG-037|Staffroom NG-037|OFFICE|105|60|14|14
NG-038|Staffroom NG-038|OFFICE|120|60|14|14
NG-039|Staffroom NG-039|OFFICE|135|60|14|14
NG-040|Staffroom NG-040|OFFICE|150|60|14|14
NG-041|Staffroom NG-041|OFFICE|165|60|14|14
NG-042|Staffroom NG-042|OFFICE|180|60|14|14
NG-043|Staffroom NG-043|OFFICE|195|60|14|14

# ----- NG-052..NG-044 : SAME ROW AS NG-030, placed to the RIGHT in
DESCENDING IDs -----
NG-052|Staffroom NG-002|OFFICE|24|42|14|14
NG-051|Staffroom NG-003|OFFICE|39|42|14|14
NG-050|Staffroom NG-004|OFFICE|54|42|14|14
NG-049|Staffroom NG-005|OFFICE|69|42|14|14
NG-048|Staffroom NG-006|OFFICE|84|42|14|14
NG-047|Staffroom NG-007|OFFICE|99|42|14|14
NG-046|Staffroom NG-008|OFFICE|114|42|14|14
NG-045|Staffroom NG-009|OFFICE|129|42|14|14
NG-044|Staffroom NG-010|OFFICE|144|42|14|14

# ----- Toilets NGT6 / NGT7 – positioned so L/R are from geometry --
-----
NGT7|Male Toilet NG|TOILET|10|90|10|10

```

```

NGT6|Female Toilet NG|TOILET|20|90|10|10

# ----- Staircases -----
STAIR_L|Staircase Left|STAIRCASE|-15|115|12|20
STAIR_R|Staircase Right|STAIRCASE|150|115|12|20

```

A.4.2 connections.txt

```

# ----- Classroom Connections -----
N007|N006|corridor
N006|N005|corridor
N005|N004|corridor
N004|N003|corridor
N003|N002|corridor
N002|N001|corridor

# ----- Lab Connections -----
N009|N010|corridor
N010|N011|corridor
N011|N012|corridor

# ----- Corridor Junctions -----
JUNC_LEFT|N007|corridor
JUNC_LEFT|N008|corridor

JUNC_MID|N004|corridor
JUNC_MID|N003|corridor
JUNC_MID|N010|corridor
JUNC_MID|N011|corridor

JUNC_RIGHT|NGT1|corridor
JUNC_RIGHT|N001|corridor

JUNC_TOP_LEFT|NG-001|corridor
JUNC_TOP_LEFT|NG-015|corridor

JUNC_BOTTOM_LEFT|NG-044|corridor
JUNC_BOTTOM_LEFT|NG-043|corridor

# ----- Toilets -----
N008|NGT3|corridor
NGT3|NGT4|corridor
NGT4|NGT5|corridor
NGT2|N002|corridor
N012|NGT1|corridor
NGT1|NGT2|corridor

```

```

# ----- Entrances -----
ENTRANCE_MAIN|JUNC_MID|entrance

# ---- Staffrooms NG-001..NG-014 (reverse order) ----
NG-014|NG-013|corridor
NG-013|NG-012|corridor
NG-012|NG-011|corridor
NG-011|NG-010|corridor
NG-010|NG-009|corridor
NG-009|NG-008|corridor
NG-008|NG-007|corridor
NG-007|NG-006|corridor
NG-006|NG-005|corridor
NG-005|NG-004|corridor
NG-004|NG-003|corridor
NG-003|NG-002|corridor
NG-002|NG-001|corridor

# ---- Staffrooms NG-015..NG-023 (positions reversed: 15 on the right, 23
on the left) ----
NG-015|NG-016|corridor
NG-016|NG-017|corridor
NG-017|NG-018|corridor
NG-018|NG-019|corridor
NG-019|NG-020|corridor
NG-020|NG-021|corridor
NG-021|NG-022|corridor
NG-022|NG-023|corridor

# ---- NG-024..NG-030 precise L-shape ----
NG-024|NG-025|corridor
NG-025|NG-026|corridor
NG-026|NG-027|corridor
NG-024|NG-023|corridor
NG-027|NG-029|corridor
NG-028|NG-030|corridor

# ---- NG-031..NG-043 : bottom row (keep standard left→right) ----
NG-031|NG-032|corridor
NG-032|NG-033|corridor
NG-033|NG-034|corridor
NG-034|NG-035|corridor
NG-035|NG-036|corridor
NG-036|NG-037|corridor
NG-037|NG-038|corridor
NG-038|NG-039|corridor
NG-039|NG-040|corridor
NG-040|NG-041|corridor

```

```

NG-041|NG-042|corridor
NG-042|NG-043|corridor

# ---- NG-052..NG-044 : SAME ROW AS NG-030, placed to the RIGHT in
DESCENDING IDs ----
NG-052|NG-051|corridor
NG-051|NG-050|corridor
NG-050|NG-049|corridor
NG-049|NG-048|corridor
NG-048|NG-047|corridor
NG-047|NG-046|corridor
NG-046|NG-045|corridor
NG-045|NG-044|corridor
NG-030|NG-052|corridor

# ---- Toilets NGT6 / NGT7 – positioned so L/R are from geometry ----
NGT6|NG-010|corridor
NGT7|NG-011|corridor
NGT6|NGT7|corridor
NGT6|NG-022|corridor

# — Staircases —
STAIR_L|NG-052|corridor
STAIR_R|NG-044|corridor

```

A.4.3 Geometry Header Source Code (Geometry.h)

```

#pragma once

namespace NavigationVI{
    struct Point{

        float distanceTo(const Point& other) const;
        float manhattanDistanceTo(const Point& other) const;

        float m_x{};
        float m_y{};
    };

    struct Rectangle{
        bool containsPoint(const Point& p) const;
        Point center() const;
        bool intersects(const Rectangle& o) const;
        float m_x{},
            m_y{},
            m_width{},
            m_height{};
    };
}

```

```
};
}
```

A.4.4 Geometry Source Code (Geometry.cpp)

```
#include "Geometry.h"

#include <cmath>

namespace NavigationVI{

    float Point::distanceTo(const Point& other) const{
        return std::hypot(m_x - other.m_x, m_y - other.m_y);
    }

    float Point::manhattanDistanceTo(const Point& other) const{
        return std::abs(m_x - other.m_x) + std::abs(m_y - other.m_y);
    }

    bool Rectangle::containsPoint(const Point& p) const{
        return (
            (m_x <= p.m_x <= m_x + m_width) &&
            (m_y <= p.m_y <= m_y + m_height)
        );
    }

    Point Rectangle::center() const{
        return Point{m_x + m_width/2, m_y + m_height/2};
    }

    bool Rectangle::intersects(const Rectangle& o) const{
        return !(
            (m_x + m_width < o.m_x) || (o.m_x + o.m_width < m_x) ||
            (m_y + m_height < o.m_y) || (o.m_y + o.m_height < m_y)
        );
    }

}
```

A.4.5 Map Entities Header Source Code (MapEntities.h)

```
#pragma once

namespace NavigationVI{
    struct Point{

        float distanceTo(const Point& other) const;
```

```

        float manhattanDistanceTo(const Point& other) const;

        float m_x{};
        float m_y{};
    };

    struct Rectangle{
        bool containsPoint(const Point& p) const;
        Point center() const;
        bool intersects(const Rectangle& o) const;
        float m_x{},
              m_y{},
              m_width{},
              m_height{};
    };
}

```

A.4.6 Map Entities Source Code (MapEntities.cpp)

```

#include "MapEntities.h"

#include <string>
#include <unordered_set>

namespace NavigationVI{
    void Room::addConnections(const std::string& roomId){
        m_connections.insert(roomId);
    }
}

```

A.4.7 Coordinate Map System Header Source Code (CoordinateMapSystem.h)

```

#pragma once

#include <string>
#include <unordered_map>
#include <vector>
#include <unordered_set>
#include <optional>
#include <queue>
#include <chrono>
#include <algorithm>
#include <cmath>

#include "../utils/RouteTypes.h"
#include "../utils/RouteInternal.h"

```



```

#include "../utils/MapEntities.h"

namespace NavigationVI {
    class CoordinateMapSystem {
    public:
        CoordinateMapSystem(const std::string& buildingName, const
std::string& floorName);

        std::unordered_map<std::string, Room> getRooms() const; //new
        void addRoom(const Room& room);
        void addConnection(const Connection& c);
        std::vector<std::string> getNeighbours(const std::string&
roomId) const;
        std::optional<Connection> getConnection(const std::string& a,
const std::string& b) const;
        float heuristic(const std::string& a, const std::string& b)
const;
        float connectionLength(const Connection& conn) const;
        float segmentCost(const Connection& con) const;
        PathResult aStarPathFind(const std::string& startRoom, const
std::string& goalRoom);
        PathResult findShortestPath(const std::string& startRoom,
const std::string& goalRoom, bool _verbose = false);
        bool loadRoomsFromFile(const std::string& filePath);
        bool loadConnectionsFromFile(const std::string& filePath);
        std::vector<Point> stitchWayPoints(std::vector<std::string>
pathIds);
        std::optional<std::string> resolveRoomId(const std::string&
indent) const;
    private:
        std::string m_buildingName{};
        std::string m_floorName{};
        std::unordered_map<std::string, Room> m_rooms{};
        std::unordered_map<std::string, std::vector<Connection>>
m_connections{};
        int m_pushCounter{ 0 };
    };
}

```

A.4.8 Coordinate Map System Source Code (CoordinateMapSystem.cpp)

```

#include <string>
#include <unordered_map> // for converting dictionaries
#include <vector> // for converting strings
#include <unordered_set> // converting sets
#include <optional>
#include <queue>

```

```

#include <chrono>
#include <algorithm>
#include <cmath>
#include <utility> // For std::forward
#include <fstream>
#include <sstream>
#include <iostream>

#include "../utils/RouteTypes.h"
#include "../utils/MapEntities.h"
#include "../utils/RouteInternal.h"

#include "CoordinateMapSystem.h"

namespace NavigationVI {
    CoordinateMapSystem::CoordinateMapSystem(
        const std::string& buildingName,
        const std::string& floorName)
        : m_buildingName(buildingName)
        , m_floorName(floorName) {}

    std::unordered_map<std::string, Room>
CoordinateMapSystem::getRooms() const {
        return m_rooms;
    }

    void CoordinateMapSystem::addRoom(const Room& room) {
        m_rooms.emplace(room.m_id, room);
        m_connections[room.m_id];
    }

    void CoordinateMapSystem::addConnection(const Connection& c) {
        m_connections[c.fromRoom].push_back(c);

        Connection rev{
            c.toRoom,
            c.fromRoom,
            c.distance,
            c.pathwayType,
            std::vector<Point>(c.wayPoints.rbegin(),
c.wayPoints.rend()),
            c.isAccessible,
            c.width
        };

        m_connections[c.toRoom].push_back(rev);
        m_rooms[c.fromRoom].addConnections(c.toRoom);
        m_rooms[c.toRoom].addConnections(c.fromRoom);
    }
}

```

```

    }

    std::vector<std::string> CoordinateMapSystem::getNeighbours(const
std::string& roomId) const {
        std::vector<std::string> neighbours{};
        auto it{ m_connections.find(roomId) };
        if (it != m_connections.end()) {
            for (const auto& c : it->second) {
                if (c.isAccessible) neighbours.push_back(c.toRoom);
            }
        }
        return neighbours;
    }

    std::optional<Connection>
CoordinateMapSystem::getConnection(const std::string& a, const
std::string& b) const {
        auto it{ m_connections.find(a) };
        if (it != m_connections.end()) {
            for (const auto& c : it->second) {
                if (c.toRoom == b) return c;
            }
        }
        return std::nullopt;
    }

    float CoordinateMapSystem::heuristic(const std::string& a, const
std::string& b) const {
        return
m_rooms.at(a).m_center.distanceTo(m_rooms.at(b).m_center);
    }

    float CoordinateMapSystem::connectionLength(const Connection&
conn) const {
        if (conn.fromRoom.find("CORRIDOR") != std::string::npos ||
            conn.toRoom.find("CORRIDOR") != std::string::npos) return
conn.distance;

        std::vector<Point> pts{};
        pts.push_back(m_rooms.at(conn.fromRoom).m_center);
        pts.insert(pts.end(), conn.wayPoints.begin(),
conn.wayPoints.end());
        pts.push_back(m_rooms.at(conn.toRoom).m_center);

        float total{ 0.0f };
        for (size_t i{ 0 }; i < pts.size() - 1; ++i) total +=
pts[i].distanceTo(pts[i + 1]);
    }

```

```

        return total;
    }

    float CoordinateMapSystem::segmentCost(const Connection& conn)
const {
    return connectionLength(conn);
}

    std::vector<Point>
CoordinateMapSystem::stitchWayPoints(std::vector<std::string>
pathIds) {
    if (pathIds.empty()) return {};

    std::vector<Point> pts{};
    pts.push_back(m_rooms.at(pathIds[0]).m_center);

    for (size_t i{ 0 }; i < pathIds.size() - 1; ++i) {
        auto c{ getConnection(pathIds[i], pathIds[i + 1]) };
        if (!c) continue;
        pts.insert(pts.end(), c->wayPoints.begin(), c-
>wayPoints.end());
        pts.push_back(m_rooms.at(pathIds[i + 1]).m_center);
    }

    std::vector<Point> cleaned{};
    cleaned.push_back(pts[0]);

    for (size_t i{ 1 }; i < pts.size(); ++i) {
        if (cleaned.back().distanceTo(pts[i]) > 0.05f)
cleaned.push_back(pts[i]);
    }
    return cleaned;
}

    PathResult CoordinateMapSystem::aStarPathFind(
    const std::string& startRoom,
    const std::string& goalRoom
) {
    auto t0{ std::chrono::high_resolution_clock::now() };

    if (m_rooms.find(startRoom) == m_rooms.end() ||
        m_rooms.find(goalRoom) == m_rooms.end()) {
        auto elapsed{
            std::chrono::duration<float>(
                std::chrono::high_resolution_clock::now() - t0
            ).count()
        };
        return PathResult{ {}, 0.0f, {}, false, elapsed };
    }

```

```

    }

    if (startRoom == goalRoom) {
        auto elapsed{ std::chrono::duration<float>(
            std::chrono::high_resolution_clock::now() -
t0).count() };
        return PathResult{ {startRoom}, 0.0f,
{m_rooms.at(startRoom).m_center}, true, elapsed };
    }

    std::priority_queue<PQEntry, std::vector<PQEntry>,
std::greater<PQEntry>> openHeap{};
    std::unordered_map<std::string, Node> openMap{};
    std::unordered_set<std::string> closed{};

    float h0{ heuristic(startRoom, goalRoom) };
    openMap[startRoom] = Node{ startRoom, 0.0f, h0, std::nullopt
};

    openHeap.push(PQEntry{ h0, h0, ++m_pushCounter, startRoom });

    while (!openHeap.empty()) {
        std::string uId{ openHeap.top().m_nodeId };
        openHeap.pop();

        if (closed.count(uId)) continue;

        const Node& u{ openMap[uId] };
        closed.insert(uId);

        if (uId == goalRoom) {
            std::vector<std::string> path{};
            std::optional<std::string> cur{ uId };
            while (cur.has_value()) {
                path.push_back(cur.value());
                cur = openMap[cur.value()].getParent();
                // cur = openMap[cur.value()].m_parent;
            }
            std::reverse(path.begin(), path.end());

            auto elapsed{
                std::chrono::duration<float>(
                    std::chrono::high_resolution_clock::now() -
t0).count()
            };
            return PathResult{ path, u.getG(),
stitchWayPoints(path), true, elapsed };
        }
    }

```

```

        for (const auto& vId : getNeighbours(uId)) {
            if (closed.count(vId)) continue;

            auto conn{ getConnection(uId, vId) };
            if (!conn || !conn->isAccessible) continue;

            float tentativeG{ u.getG() +
segmentCost(conn.value()) };

            auto it{ openMap.find(vId) };
            if (it == openMap.end() || tentativeG < it-
>second.getG() - 1e-12f) {
                float h{ heuristic(vId, goalRoom) };
                openMap[vId] = Node(vId, tentativeG, h, uId);
                float f{ tentativeG + h };
                openHeap.push(PQEntry{ f, h, ++m_pushCounter, vId
});
            }
        }

        auto elapsed{
            std::chrono::duration<float>{
                std::chrono::high_resolution_clock::now() - t0
            }.count() };

        return PathResult{ {}, 0.0f, {}, false, elapsed };
    }

    PathResult CoordinateMapSystem::findShortestPath(const
std::string& startRoom, const std::string& goalRoom, bool _verbose) {
        auto sId{ resolveRoomId(startRoom) };
        auto gId{ resolveRoomId(goalRoom) };

        if (!sId || !gId) return PathResult{ {}, 0.0f, {}, false,
0.0f };

        return aStarPathFind(sId.value(), gId.value());
    }

    std::optional<std::string>
CoordinateMapSystem::resolveRoomId(const std::string& ident) const {
        if (m_rooms.find(ident) != m_rooms.end()) return ident;
        std::string lowerIdent{ ident };
        std::transform(
            lowerIdent.begin(),
            lowerIdent.end(),

```

```

        lowerIdent.begin(),
        ::tolower);

    for (const auto& [rId, r] : m_rooms) {
        std::string lowerName = r.m_name;
        std::transform(
            lowerName.begin(),
            lowerName.end(),
            lowerName.begin(),
            ::tolower);
        if (lowerName == lowerIdent) return rId;
    }
    return std::nullopt;
}

bool CoordinateMapSystem::loadRoomsFromFile(const std::string&
filePath) {
    std::ifstream file(filePath);
    if (!file.is_open()) return false;

    std::string line{};

    while (std::getline(file, line)) {
        if (line.empty() || line[0] == '#') continue; // skip
comments

        std::stringstream ss(line);
        std::string id{}, name{}, typeStr{};
        float x{}, y{}, w{}, h{};
        char delim{};

        std::getline(ss, id, '|');
        std::getline(ss, name, '|');
        std::getline(ss, typeStr, '|');

        auto trim{ [](std::string& s) {
            s.erase(s.begin(), std::find_if(s.begin(), s.end(),
                [](unsigned char ch) {return !std::isspace(ch);
            }));
            s.erase(std::find_if(s.rbegin(), s.rend(),
                [](unsigned char ch) {return !std::isspace(ch);
            }).base(), s.end());
        } };
        trim(typeStr);
        std::transform(typeStr.begin(), typeStr.end(),
typeStr.begin(),
            [](unsigned char c) {return std::toupper(c); });

```

```

        ss >> x >> delim >> y >> delim >> w >> delim >> h;

        Room r{};
        r.m_id = id;
        r.m_name = name;

        try {
            r.m_RoomType = roomTypeFromString(typeStr);
        }
        catch (const std::invalid_argument& e)
        {
            std::cerr << "Skipping room due to invalid type: " <<
e.what() << "\n";
            continue;
        }

        r.m_bounds = Rectangle{ x, y, w, h };
        r.m_center.m_x = x;
        r.m_center.m_y = y;
        addRoom(r);
    }

    return true;
}

bool CoordinateMapSystem::loadConnectionsFromFile(const
std::string& filePath) {
    std::ifstream file(filePath);
    if (!file.is_open()) return false;

    std::string line{};

    while (std::getline(file, line)) {
        if (line.empty() || line[0] == '#') continue;

        std::stringstream ss(line);
        std::string a{}, b{}, type{};

        if (!std::getline(ss, a, '|')) continue;
        if (!std::getline(ss, b, '|')) continue;
        if (!std::getline(ss, type, '|')) continue;

        Point pa{ m_rooms.at(a).m_center };
        Point pb{ m_rooms.at(b).m_center };
        float dist{ pa.distanceTo(pb) };

        Connection c{ a, b, dist, type, {}, true, 0.0f };
        addConnection(c);
    }
}

```



```

        return true;
    }
}

```

A.4.9 Route Guidance Header Source Code (RouteGuidance.h)

```

#pragma once

#include <string>
#include <vector>
#include <map>
#include <set>
#include <optional>
#include <functional>

#include "../utils/MapEntities.h"
#include "../utils/RouteTypes.h"
#include "CoordinateMapSystem.h"

namespace NavigationVI {
    struct Instruction {
        std::string text{};
        double distance_m{ 0.0f };
        int steps{ 0 };
        Instruction() = default;
        Instruction(const std::string& t, double d = 0.0, int s = 0)
            : text(t), distance_m(d), steps(s) {}
    };

    class RouteGuidance {
    public:
        RouteGuidance() = default;

        std::pair<std::vector<Instruction>, std::map<std::string,
double>>>
            pathToInstructions(CoordinateMapSystem map,
                const std::string& startRoom,
                const std::string& goalRoom,
                double unitScale = 1.0,
                double stepLengthM = 0.75,
                const std::string& mode = "step",
                double landmarkRadius = 20.0,
                bool anchorEverySegment = true
            );

        double estimateStrideFromHeightCm(double height_cm) const;
    };
}

```

```

    public:
        std::function<void(const std::string&)> onMessage{};

    private:
        std::pair<double, double> pointSegmentDistance(const Point&
p, const Point& a, const Point& b) const;
        std::string sideOfPoint(const Point& p, const Point& a, const
Point& b, double eps = 1e-6) const;
        std::optional<Room> roomAtPoint(const Point& p, const
CoordinateMapSystem& map, double tol = 1e-5) const;
        double bearingDeg(const Point& a, const Point& b) const;
        std::string turnPhrase(std::optional<double> prevBearing,
double currBearing) const;
        double segmentDistanceM(const Point& a, const Point& b,
double unitScale) const;
        double calibrateUnitScaleFromSteps(const std::string& aRoom,
const std::string& bRoom,
            int steps, const CoordinateMapSystem& map, double
stepLengthM = 0.75) const;
    private:
        std::map<std::string, Room> rooms{};
};
}

```

A.4.10 Route Internal Header Source Code (RouteInternal.h)

```

#pragma once

#include <string>
#include <optional>

namespace NavigationVI{
    class PQEntry{
    public:
        PQEntry(const float f,
                const float h,
                const int counter,
                const std::string& nodeId)
            : m_f(f)
            , m_h(h)
            , m_counter(counter)
            , m_nodeId(nodeId){}

        bool operator>(const PQEntry& other) const {
            if (m_f == other.m_f){
                if (m_h == other.m_h) return m_counter >
other.m_counter;

```

```

        return m_h > other.m_h;
    }
    return m_f > other.m_f;
}

public:
    std::string m_nodeId{};
private:
    int m_counter{};
    float m_f{};
    float m_h{};
};

class Node{
public:
    Node() : m_roomId(""), m_g(0.0f), m_h(0.0f),
m_parent(std::nullopt){}
    Node(const std::string& roomId,
        const float g,
        const float h,
        const std::optional<std::string> parent)
        : m_roomId (roomId)
        , m_g (g)
        , m_h (h)
        , m_parent(parent){}

    std::optional<std::string> getParent() const {return
m_parent; }
    float getG() const {return m_g;}
    float getH() const {return m_h;}
private:
    std::string m_roomId{};
    float m_g{};
    float m_h{};
    std::optional<std::string> m_parent{};
};
}

```

A.4.11 Route Type Header Source Code (RouteType.h)

```

#pragma once

#include <vector>
#include <string>

#include "Geometry.h"

```

```

namespace NavigationVI{
    struct PathResult{
        std::vector<std::string> m_path{};
        float m_totalDistance{};
        std::vector<Point> m_wayPoints{};
        bool m_found{};
        float m_executionTime{};
    };
}

```

A.4.12 Route Guidance Source Code (RouteGuidance.cpp)

```

#include <cmath>
#include <algorithm>
#include <limits>
#include <iostream>
#include <sstream>

#include "CoordinateMapSystem.h"
#include "RouteGuidance.h"
#include "../utils/Geometry.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

namespace NavigationVI {
    std::pair<double, double> RouteGuidance::pointSegmentDistance(
        const Point& p, const Point& a, const Point& b) const {
        double vx{ b.m_x - a.m_x };
        double vy{ b.m_y - a.m_y };
        double wx{ p.m_x - a.m_x };
        double wy{ p.m_y - a.m_y };
        double denom{ vx * vx + vy * vy };
        if (denom < 1e-12) return { a.distanceTo(p), 0.0 };
        double t{ std::max(0.0, std::min(1.0, (wx * vx + wy * vy) /
denom)) };
        Point proj{};
        proj.m_x = a.m_x + t * vx;
        proj.m_y = a.m_y + t * vy;
        return { proj.distanceTo(p), t };
    }

    std::string RouteGuidance::sideOfPoint(const Point& p, const
Point& a, const Point& b, double eps) const {
        double cross{ (b.m_x - a.m_x) * (p.m_y - a.m_y) - (b.m_y -
a.m_y) * (p.m_x - a.m_x) };

```

```

        if (cross > eps) return "left";
        if (cross < -eps) return "right";
        return "ahead";
    }

    std::optional<Room> RouteGuidance::roomAtPoint(const Point& p,
const CoordinateMapSystem& map, double tol) const {
        for (const auto& kv : map.getRooms()) {
            if (kv.second.m_center.distanceTo(p) <= tol) return
kv.second;
        }
        return std::nullopt;
    }

    double RouteGuidance::bearingDeg(const Point& a, const Point& b)
const { return std::atan2(b.m_y - a.m_y, b.m_x - a.m_x) * 180.0 /
M_PI; }

    std::string RouteGuidance::turnPhrase(std::optional<double>
prevBearing, double currBearing) const {
        if (!prevBearing.has_value()) return "Head";

        // Signed smallest difference in degrees, normalized to
(-180, 180]
        double diff = currBearing - prevBearing.value();
        diff = std::fmod(diff + 540.0, 360.0) - 180.0; // normalize

        double ad = std::fabs(diff);
        if (ad < 15) return "Continue straight";
        if (ad < 135) return diff > 0 ? "Turn left" : "Turn right";
        return "Make a U-turn";
    }

    double RouteGuidance::segmentDistanceM(const Point& a, const
Point& b, double unitScale) const { return a.distanceTo(b) *
unitScale; }

    double RouteGuidance::estimateStrideFromHeightCm(double
height_cm) const { return 0.414 * (height_cm / 100.0); }

    double RouteGuidance::calibrateUnitScaleFromSteps(
        const std::string& aRoom,
        const std::string& bRoom,
        int steps,
        const CoordinateMapSystem& map,
        double stepLengthM) const {

```

```

        if (!map.getRooms().count(aRoom) ||
!map.getRooms().count(bRoom) || steps <= 0) return 1.0;
        double mapUnits{
map.getRooms().at(aRoom).m_center.distanceTo(map.getRooms().at(bRoom)
.m_center) };
        double real_m{ steps * stepLengthM };
        return real_m / std::max(mapUnits, 1e-9);
    }

    std::pair<std::vector<Instruction>, std::map<std::string,
double>>
RouteGuidance::pathToInstructions(CoordinateMapSystem map,
    const std::string& startRoom,
    const std::string& goalRoom,
    double unitScale,
    double stepLengthM,
    const std::string& mode,
    double landmarkRadius,
    bool anchorEverySegment)
{

    std::vector<Instruction> instrs{};
    std::map<std::string, double> summary;

    auto result{ map.findShortestPath(startRoom, goalRoom) };
    if (!result.m_found || result.m_path.empty()) {
        instrs.emplace_back("No path found from" + startRoom + "
to " + goalRoom + ".");
        summary["found"] = 0.0;
        summary["total_m"] = 0.0;
        summary["total_steps"] = 0.0;
        summary["segment"] = 0.0;
        return { instrs, summary };
    }

    std::vector<Point> pts{ map.stitchWayPoints(result.m_path) };
    if (pts.empty()) {
        instrs.emplace_back("No waypoints for path.");
        summary["found"] = 0.0;
        return { instrs, summary };
    }

    std::vector<Point> dedup{};
    dedup.push_back(pts.front());
    for (size_t i{ 1 }; i < pts.size(); ++i) {
        if (dedup.back().distanceTo(pts[i]) > 1e-6)
dedup.push_back(pts[i]);
    }
}

```

```

        pts.swap(dedup);

        double total_m{ 0.0f };
        int total_steps{ 0 };
        std::optional<double> prev_bearing{};

        std::set<std::string> excludeLandmarks(result.m_path.begin(),
result.m_path.end());
        std::string startName{
map.getRooms().at(result.m_path.front()).m_name };
        std::string goalName{
map.getRooms().at(result.m_path.back()).m_name };

        instrs.emplace_back("Starting at " + startName + ".");
        if (onMessage) onMessage("Starting at " + startName + ".");

        for (size_t i{ 0 }; i + 1 < pts.size(); ++i) {
            const Point& a{ pts[i] };
            const Point& b{ pts[i + 1] };

            double seg_m_if_scaled{ a.distanceTo(b) * unitScale };
            int seg_steps{};
            if (unitScale != 1.0) seg_steps = std::max(1,
(int)std::lround(seg_m_if_scaled / std::max(stepLengthM, 1e-6)));
            else seg_steps = std::max(1,
(int)std::lround(a.distanceTo(b) / std::max(stepLengthM, 1e-6)));

            double approx_m{ seg_steps * stepLengthM };

            double bearing{ bearingDeg(a, b) };
            std::string action{ turnPhrase(prev_bearing, bearing) };
            prev_bearing = bearing;

            // Anchor to next node name if we're at a room vertex
            std::string at_phrase{};
            auto b_node{ roomAtPoint(b, map) };
            if (b_node.has_value() && (anchorEverySegment || action
!= "Continue straight")) {
                at_phrase = " to " + b_node->m_name;
            }

            // Distance phrase (no landmark mode anymore)
            std::string distance_phrase{};
            if (mode == "map") {
                std::ostringstream oss{};
                oss << " for about " <<
(int)std::lround(seg_m_if_scaled) << " meters";

```

```

        distance_phrase = oss.str();
    }
    else if (mode == "steps") {
        std::ostringstream oss{};
        oss << " for about " << seg_steps << " steps (~ " <<
(int)std::lround(approx_m) << " m)";
        distance_phrase = oss.str();
    }
    else {
        // default: same as steps (choose what you prefer)
        std::ostringstream oss{};
        oss << " for about " << seg_steps << " steps (~ " <<
(int)std::lround(approx_m) << " m)";
        distance_phrase = oss.str();
    }

    // Final instruction (NO landmark_phrase)
    std::string text{ action + at_phrase + distance_phrase +
"." };

    instrs.emplace_back(text, approx_m, seg_steps);
    if (onMessage) onMessage(text);
    total_m += approx_m;
    total_steps += seg_steps;
}

instrs.emplace_back("Arrive at " + goalName + ".");
if (onMessage) onMessage("Arrive at " + goalName + ".");
summary["found"] = 1.0;
summary["total_m"] = total_m;
summary["total_steps"] = total_steps;
summary["segment"] = static_cast<double>(std::max(0,
(int)pts.size() - 1));
summary["unit_scale"] = unitScale;
summary["step_length_m"] = stepLengthM;

return { instrs, summary };
}
}

```

A.4.13 Text To Speech Header Source Code (TextToSpeech.h)

```

#pragma once

#include <string>

namespace NavigationVI {

```



```

class TextToSpeech {
public:
    static std::string platformPause(int ms);
    void speak(const std::string& text);
};
}

```

A.4.14 Text To Speech Source Code (TextToSpeech.cpp)

```

#include "TextToSpeech.h"

#include <cstdlib>

namespace NavigationVI {
    std::string TextToSpeech::platformPause(int ms) {
#ifdef _WIN32
        // SAPI supports SSML <break>
        return "<break time='" + std::to_string(ms) + "ms' />";
#elif __APPLE__
        // macOS 'say' doesn't support SSML or slnc – just return
        empty
        return "";
#else
        // eSpeak pause syntax
        return "[[slnc " + std::to_string(ms) + "]]";
#endif
    }
    void TextToSpeech::speak(const std::string& text) {
#ifdef _WIN32
        std::string command = "PowerShell -Command \"Add-Type -
        AssemblyName System.Speech; \"
        \"(New-Object
        System.Speech.Synthesis.SpeechSynthesizer).Speak('" + text + "');\"";
#elif __APPLE__
        std::string command = "say \"" + text + "\"";
#else
        std::string command = "espeak \"" + text + "\"";
#endif
        std::system(command.c_str());
    }
}

```

APPENDIX B

PLAGIARISM CHECK RESULT

15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 8 words)

Match Groups

-  **199** Not Cited or Quoted 15%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 10%  Internet sources
- 3%  Publications
- 14%  Submitted works (Student Papers)

APPENDIX B

Match Groups

-  **199 Not Cited or Quoted 15%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 10%  Internet sources
- 3%  Publications
- 14%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Universiti Tunku Abdul Rahman on 2021-08-25	5%
2	Internet	eprints.utar.edu.my	1%
3	Student papers	FICT on 2025-08-14	<1%
4	Student papers	Universiti Tunku Abdul Rahman on 2023-09-11	<1%
5	Student papers	Universiti Tunku Abdul Rahman on 2022-04-20	<1%
6	Student papers	FICT on 2025-08-11	<1%
7	Student papers	FICT on 2025-08-14	<1%
8	Student papers	UC, Boulder on 2025-08-13	<1%
9	Student papers	FICT on 2025-08-12	<1%
10	Student papers	FICT on 2025-08-12	<1%

APPENDIX B

11	Student papers	Universiti Tunku Abdul Rahman on 2023-09-11	<1%
12	Internet	mmocr.readthedocs.io	<1%
13	Student papers	FICT on 2025-08-14	<1%
14	Student papers	Universiti Tunku Abdul Rahman on 2024-09-20	<1%
15	Student papers	University of Gloucestershire on 2025-09-15	<1%
16	Student papers	FICT on 2025-08-11	<1%
17	Student papers	Universiti Tunku Abdul Rahman on 2023-09-11	<1%
18	Publication	de Oliveira Cadete, Tomás. "Dynamic AMR Navigation with Trajectory Prediction ...	<1%
19	Student papers	Universiti Tenaga Nasional on 2013-09-09	<1%
20	Student papers	University of Glasgow on 2024-09-02	<1%
21	Internet	www.twblogs.net	<1%
22	Internet	git.sdf.org	<1%
23	Student papers	Cranfield University on 2025-02-24	<1%
24	Student papers	FICT on 2025-08-14	<1%

APPENDIX B

25	Student papers	University College Dublin (UCD) on 2024-06-25	<1%
26	Internet	git.lubuntu.me	<1%
27	Internet	www.mdpi.com	<1%
28	Student papers	University of Nottingham on 2023-03-19	<1%
29	Student papers	FICT on 2025-09-18	<1%
30	Internet	e2e.ti.com	<1%
31	Student papers	Asia Pacific Institute of Information Technology on 2024-06-23	<1%
32	Student papers	Cranfield University on 2025-02-24	<1%
33	Student papers	Universiti Tunku Abdul Rahman on 2022-09-09	<1%
34	Student papers	Universiti Teknikal Malaysia Melaka on 2025-06-30	<1%
35	Student papers	University of Southern California on 2020-03-22	<1%
36	Internet	git.someserver.de	<1%
37	Student papers	Godalming College on 2024-01-11	<1%
38	Student papers	Universiti Tunku Abdul Rahman on 2023-09-10	<1%

APPENDIX B

39	Internet	cpc110.blogspot.com	<1%
40	Publication	Funk, Dylan J.. "Particle Charge Determination in a Magnetized Dusty Plasma Flo...	<1%
41	Student papers	University of Bath on 2017-05-05	<1%
42	Student papers	University of Nottingham on 2025-04-07	<1%
43	Internet	answers.opencv.org	<1%
44	Student papers	Cornell University on 2023-12-14	<1%
45	Student papers	Swinburne University of Technology on 2024-11-24	<1%
46	Student papers	University of Nottingham on 2025-04-29	<1%
47	Student papers	The University of Manchester on 2020-05-01	<1%
48	Student papers	Universiti Tunku Abdul Rahman on 2015-04-05	<1%
49	Internet	lists.lre.epita.fr	<1%
50	Student papers	FICT on 2025-09-12	<1%
51	Student papers	Mindanao State University - IIT on 2021-12-26	<1%
52	Student papers	Southern New Hampshire University - Continuing Education on 2025-08-24	<1%

APPENDIX B

53	Student papers	Universiti Tunku Abdul Rahman on 2024-09-19	<1%
54	Student papers	University of Sydney on 2021-04-18	<1%
55	Internet	www.cnblogs.com	<1%
56	Student papers	Cranfield University on 2025-02-24	<1%
57	Publication	Gao, Zhaolin. "Development and Characterization of Novel Geopolymer-Based Art...	<1%
58	Publication	Haiying Yang. "Mining and Evaluation of Teaching Civics Courses Based on Aprior...	<1%
59	Publication	Jiequan Li, Tong Zhang, Shuli Yang. "The two-dimensional Riemann problem in ga...	<1%
60	Student papers	Universiti Tunku Abdul Rahman on 2024-09-19	<1%
61	Publication	Karla Holmboe. "Frontal Cortex Functioning in Infancy", Thesis Commons, 2017	<1%
62	Publication	T. Tamilarasi, P. Muthulakshmi, Seyed-Hassan Miraei Ashtiani. "Smart Edge Comp...	<1%
63	Student papers	Universiti Tunku Abdul Rahman on 2013-04-10	<1%
64	Internet	dione.lib.unipi.gr	<1%
65	Internet	discovery.dundee.ac.uk	<1%
66	Internet	uvadoc.uva.es	<1%

APPENDIX B

67	Student papers	Buckinghamshire Chilterns University College on 2016-05-11	<1%
68	Student papers	Cranfield University on 2025-02-05	<1%
69	Student papers	LASALLE-SIA College of the Arts on 2022-02-20	<1%
70	Student papers	UC, Boulder on 2025-06-28	<1%
71	Student papers	Universiti Tunku Abdul Rahman on 2013-08-12	<1%
72	Student papers	Universiti Tunku Abdul Rahman on 2017-08-29	<1%
73	Student papers	Universiti Tunku Abdul Rahman on 2019-08-23	<1%
74	Student papers	Universiti Tunku Abdul Rahman on 2023-04-25	<1%
75	Student papers	Universiti Tunku Abdul Rahman on 2023-04-27	<1%
76	Student papers	Universiti Tunku Abdul Rahman on 2024-08-21	<1%
77	Student papers	University of Nottingham on 2025-04-06	<1%
78	Student papers	AUT University on 2024-10-18	<1%
79	Student papers	Aberystwyth University on 2016-05-03	<1%
80	Publication	Arvind Dagur, Dharendra Kumar Shukla, Nazarov Fayzullo Makhmadiyarovich, Ak...	<1%

APPENDIX B

81	Student papers	Consortio CIXUG on 2020-09-10	<1%
82	Publication	Evianita Dewi Fajrianti, Amma Liesvarastranta Haz, Nobuo Funabiki, Sri trusta Suk...	<1%
83	Student papers	FICT on 2025-08-19	<1%
84	Publication	Fatma A. Mihany, Galal H. Galal-Edeen, Ehab E. Hassanein, Hanan Moussa. "Data-...	<1%
85	Student papers	King's College on 2022-08-03	<1%
86	Publication	Md. Milon Islam, Muhammad Sheikh Sadi, Kamal Z. Zamli, Md. Manjur Ahmed. "D...	<1%
87	Publication	Poonam Nandal, Mamta Dahiya, Meeta Singh, Arvind Dagur, Brijesh Kumar. "Pro...	<1%
88	Publication	Sandeep Babu, Suja Markose. "IoT Enabled Robots with QR Code Based Localizati...	<1%
89	Publication	Shams Al Ajrawi, Charity Jennings, Paul Menefee, Wathiq Mansoor, Mansoor Ahm...	<1%
90	Publication	Tagliavini Ponce, Guido. "The Power of Low Associativity", Rutgers The State Univ...	<1%
91	Student papers	Universiti Tunku Abdul Rahman on 2023-04-27	<1%
92	Student papers	Universiti Tunku Abdul Rahman on 2024-08-20	<1%
93	Student papers	Universiti Tunku Abdul Rahman on 2024-09-24	<1%
94	Student papers	Universiti Tunku Abdul Rahman on 2025-05-09	<1%

APPENDIX B

95	Student papers	University of Nottingham on 2025-03-17	<1%
96	Student papers	University of Southampton on 2023-09-13	<1%
97	Student papers	University of Sydney on 2024-11-13	<1%
98	Student papers	University of Sydney on 2025-05-04	<1%
99	Publication	de Oliveira Eusébio, José Luís. "Interface virtual com sensor 3D para interação de ...	<1%
100	Internet	www.caa.co.uk	<1%